

1. [Welcome and Hypothesis](#)
2. [Tech Stack and Libraries](#)
3. [The Big Picture](#)
4. [Knowledge Base and Indexing](#)
5. [Layered Retrieval Pipeline](#)
6. [The Cross Encoder Reranker](#)
7. [The State Schema](#)
8. [The LangGraph State Machine](#)
9. [The Six Cognitive Nodes](#)
10. [The Verification Node](#)
11. [Multi Provider LLM Layer](#)
12. [Multi Turn Chat Memory](#)
13. [Guided Examples](#)
14. [The Streamlit Frontend](#)
15. [The Evaluation Suite](#)
16. [LLM as Judge Framework](#)
17. [Human Spot Check](#)
18. [Four Configurations and Ablations](#)
19. [Real Evaluation Results](#)
20. [Verbatim Prompts](#)
21. [Every Helper, Every Default](#)
22. [Decisions and Trade Offs](#)
23. [Test Suite](#)
24. [File by File Reference](#)
25. [Interview Cheat Sheet](#)
26. [Glossary](#)

Multi Provider LLMs

The ashGPT Codebase, Explained End

to End

A guided tour of every part of this multi node LangGraph agent.

Architecture, libraries, retrieval pipeline, prompts, evaluation, ablations, and

worked examples that show the system reasoning over Australian property

law.

Welcome

ashGPT is an experimental retrieval augmented agent that helps a law student study Australian property law. It is built around a single research question. Can a multi node agent that pulls chronological facts apart from binding legal rules produce more accurate causal analyses than a single prompt summariser, when both see the exact same retrieved evidence. The repository sets up the pipeline, the data, the layered retrieval stack, the post synthesis verification node, the evaluation harness, and the frontend needed to answer that question with measurable evidence.

| **Design hypothesis.** Cognitive separation versus attention dilution. A multi
| node LangGraph state machine that separates ratio decidendi extraction from
| chronological fact extraction will produce more accurate causal analyses than
| a single consolidated mega prompt baseline that receives identical retrieved
| context.

The system is intentionally compact. Six cognitive nodes including a post synthesis verification stage, one ChromaDB collection with metadata filters, a cross encoder reranker that re scores the diverse MMR pool, three model providers behind a single dispatcher, and a twenty two case evaluation suite that scores four configurations head to head. Every architectural choice in the project flows from the hypothesis. The router exists so that ratio and chronology can be measured independently. The dual index exists so that lecture slides count as evidence on the same footing as PDF readings. The reranker exists to lift the most relevant chunks above the diversity driven candidates. The verification node exists to catch any case citation that survived synthesis but is not anchored in the retrieved chunks. The two stage cross provider judge exists so that the model that wrote the answer never grades it alone.

What changed since the first draft

The codebase has grown beyond the original five node design. The new pieces are all observable in the repository.

- A **cross encoder reranker** sits after MMR. `src/agent/reranker.py` wraps `cross-encoder/ms-marco-MiniLM-L-6-v2`. `tools.py` over fetches with MMR and reduces with the cross encoder.
- A **verification node** runs after synthesis. `src/agent/verification.py` extracts every plausible case citation from the final answer and checks it against the retrieved chunk text. `nodes.py` hosts the node itself. The compiled graph wires it after synthesis when `USE_VERIFICATION` is on.
- The eval harness now compares **four configurations**. Plain LLM Baseline, Mega Prompt Ablation, Agent without reranker, and the Full Agent. The fourth column isolates the reranker's contribution.
- The eval suite grew from fifteen cases to **twenty two cases**. Six factual, five cross modal, seven analytical, four conversational including a three turn case.
- **IRAC compliance is now intent conditional**. Chronology and general queries are scored as not applicable rather than zero, which is fairer to the agent.
- A small **human spot check log** at `eval_results/human_spot_check.md` validates the LLM judge against five sampled cases. Five of five exact agreement on this dataset.

How to read this textbook

The sidebar lists every section. You can read it linearly as a tour or jump to a specific concern. The early sections build mental models. The middle sections walk through the code path of a single user message including the verification stage. The late sections cover evaluation, the four way ablation, and the test suite. A file by file reference at the end maps every section back to the line of code it explains.

Tech Stack and Libraries

Each library in the project has a well defined role. Knowing what each one is responsible for makes the code far easier to read.

Layer	Library	Responsibility in ashGPT
Orchestration	langgraph	Builds the conditional state machine. Compiles nodes and edges into a single executable graph. Handles state merging when nodes return partial dicts.
RAG plumbing	langchain, langchain-chroma	Provides the Chroma vector store wrapper, document objects, and the recursive text splitter used during indexing.
Vector store	chromadb	Local persistent store under chroma_db/. One collection holds both PDF text chunks and VLM described slide chunks, distinguished by metadata.
Embeddings	ZeroEntropy zembed-1	An asymmetric embedding model. Documents and queries are embedded with different input modes. Wrapped by hand because LangChain has no built in client.
Reranker	sentence-transformers with cross-encoder/ms-marco-MiniLM-L-6-v2	A cross encoder that scores each query and chunk pair after MMR. Loaded lazily as a module level singleton so the first call pays the load cost and every subsequent call reuses the cached instance.
PDF extraction	pymupdf (fitz)	Pulls raw text from each PDF page during indexing. Faster and more accurate than pypdf for this content.
VLM	Google gemini-2.5-pro	Reads slide images at index time and

Layer	Library	Responsibility in ashGPT
		writes a verbose plain text description that gets embedded. This is what allows lecture slides to be retrieved by a text query.
Reasoning models	Google Gemini, OpenAI GPT, Anthropic Claude	Each node may use a different provider. A custom dispatcher in <code>src/llm.py</code> routes by model name prefix.
Frontend	streamlit, streamlit-mermaid	Renders the chat UI, the trace banner, and inline interactive Mermaid diagrams. Stores per tab session memory.
Evaluation	matplotlib	Generates the PNG plots that compare configurations. Uses the Agg backend so the script works headless.
Tests	pytest	Two tier suite. Unit tests run without API keys. Integration tests use the integration marker so they can be skipped.

Why three providers. Because the design must avoid same model self evaluation bias. The agent uses Gemini in some nodes and OpenAI in others, and the judge uses Gemini for its first pass and OpenAI for its second. If the system used only one vendor, the conclusions would have a credibility problem. The dispatcher in `src/llm.py` hides this from every other module. The reranker sits outside this story because a small cross encoder is cheap, deterministic, and runs locally with no API key, which makes it a stable layer to depend on.

The Big Picture

Before diving into individual files, look at the system end to end. Two things happen offline once. Three things happen on every user message. A fourth thing happens on demand during evaluation.

flowchart LR subgraph offline ["Offline. Run once"] P[PDF readings and

tutorials] --> E[PyMuPDF text extract] E --> S[Recursive splitter. 1500 chars,

300 overlap] S --> ZE[ZeroEntropy zembed-1 document mode] Lec[JPEG and

PNG slides] --> VLM[Gemini 2.5 Pro VLM. Describe slide] VLM -->

ZS[ZeroEntropy zembed-1 document mode] ZE --> Chroma[(ChromaDB.

property_law_kb)) ZS --> Chroma end subgraph online ["Online. Per user

message"] Q[User query plus chat history] --> R{Router} R -->|intent and

week filter| Ret[Retrieval. MMR over Chroma, over fetch] Ret --> RR[Cross

Encoder Reranker] RR --> Branch{Conditional edge} Branch -->|ratio|

RX[Ratio Extractor] Branch -->|chronology| CG[Chronology Generator] Branch

-->|summary| RX Branch -->|general| Syn[Synthesis] RX -->|if summary| CG

```
RX -->|if ratio| Syn[Synthesis] CG --> Syn Syn --> V[Verification. Citation
```

```
regex + rewrite] V --> UI[Streamlit. Render answer plus inline Mermaid] end
```

```
Chroma -->|read at runtime| Ret
```

The offline phase produces a single ChromaDB collection on disk. Every chunk in that collection carries metadata that says which week it belongs to, which document type it came from, and where the source file lives. From that point forward, the agent never touches the original PDFs or images at runtime. All retrieval is metadata aware.

The online phase is a single LangGraph invocation. The router reads the query plus prior chat turns, classifies the intent, and may pull a week filter out of the wording. Retrieval over fetches via MMR, then a cross encoder reranker re scores each candidate against the original query and truncates to the top eight text chunks and four slides. After retrieval the graph forks based on intent. The forks merge at synthesis, which always runs and grounds its answer against the retrieved sources. After synthesis a verification node extracts every case citation in the final answer, checks the retrieved chunks for support, and asks synthesis to rewrite if any citation is unsupported.

Knowledge Base and Indexing

The knowledge base is the foundation of every retrieval call. It is built once by `src/indexing/build_index.py` and then read at every query.

Folder layout

data/

```
|— week_1/
| |— lecture/  JPEG or PNG slide images
| |— readings/  PDF readings
| |— tutorial/  PDF tutorials
|— week_2/
| |— ...
|— (optional root level supplementary PDFs)
```

The indexer walks every week_* directory in order, then walks each subdirectory inside. Two ingestion routines exist. _ingest_pdfs handles text. _ingest_slides handles images.

Text ingestion

Every PDF is opened with PyMuPDF. The pages are joined with a double newline so the chunker can use paragraph boundaries. The RecursiveCharacterTextSplitter from LangChain then breaks the text into 1500 character pieces with 300 character overlap. Why such a large chunk. Legal text is dense. A holding can take half a page. Smaller chunks split sentences across embeddings and retrieval pulls the wrong slices. The 300 character overlap protects against that exact failure for paragraph borders.

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE,      # 1500
    chunk_overlap=CHUNK_OVERLAP, # 300
    separators=["\n\n", "\n", ". ", " ", ""],
)
```

Example. Splitter in action

Input

A 4200 character PDF body extracted from

readings_week_3_adverse_possession.pdf.

Output

Three chunks. Chunk 0 is characters 0 to 1500. Chunk 1 is characters 1200 to

2700. Chunk 2 is characters 2400 to 4200. The 300 character overlap at chunk

borders keeps split sentences readable in both halves.

Code link. The splitter call is in `build_index.py`. The constants `CHUNK_SIZE`

and `CHUNK_OVERLAP` live in `src/config.py`. Separators try paragraph break

first, then newline, then sentence end, then word, then character. Whichever

holds the chunk size constraint wins.

Each chunk gets a deterministic ID built from the week, the document type, the file stem, and the chunk index. That ID rule means re running the indexer never produces duplicates. It overwrites the same key.

Example. Deterministic IDs and metadata

Input

Filename Sackville_Ch2_Adverse_Possession.pdf in data/week_3/readings/,

splits into 6 chunks.

Output

IDs week_3_reading_Sackville_Ch2_Adverse_Possession_chunk_0 through

_chunk_5. Each carries metadata {"week": "week_3", "type": "reading",

"source": "Sackville_Ch2_Adverse_Possession.pdf", "chunk_index": 0..5}.

Code link. See `_sanitise_id` for the character substitution rule that turns

dots and spaces into underscores. The ID list and metadata list are passed

together to `collection.add_texts` so Chroma stores them in lockstep.

The metadata attached to every chunk is what makes filtered retrieval possible later.

week.e.g. week_3 or all for root level PDFs. typereading, tutorial, supplementary, or lecture_slide. sourceOriginal filename. chunk_indexPosition inside its source document. image_pathOnly set for slides. Points at the original image file on disk.

Slide ingestion (the cross modal trick)

The interesting part of indexing is how slide images are made retrievable. Vector stores do not embed images directly here. Instead, every slide is sent to a vision language model with a careful prompt. That model produces a long description that captures every visible piece of text, every diagram, and every case name. The description, not the image, is what gets embedded.

```
SLIDE_DESCRIPTION_PROMPT = (  
    "You are a legal education assistant. Describe this property law lecture "  
    "slide in detail. Include all visible text verbatim, any diagrams or "  
    "flowcharts, legal principles, case names, statutory references, and "  
    "definitions. Preserve the structure (headings, bullet points) as closely "  
    "as possible."  
)
```

Example. Slide image to indexed text

Input

A JPEG of a lecture slide titled "Possession of Chattels" with two bullet points

and a Moukataff v BOAC case reference.

Output

A 600 to 1200 character description like: "Heading. Possession of Chattels. Two

requirements. First, factual control over the chattel. Second, intention to

exclude others. Case reference. Moukataff v British Overseas Airways Corp

[1967] 1 Lloyd's Rep 396. Diagram description. Flow arrow from physical

custody to ownership claim..."

Then

The whole description is embedded with zembed-1 in document mode and

stored under ID week_3_lecture_slide_slide_05 with metadata {"week":

"week_3", "type": "lecture_slide", "source": "slide_05.jpg", "image_path":

"data/week_3/lecture/slide_05.jpg"}.

Code link. `describe_slide` reads bytes, builds a Gemini Part with the right

MIME, calls `generate_content`. `_ingest_slides` handles the embedding and the

metadata dict.

This is what allows a question like "what does the lecture slide say about Moukataff" to hit slides as well as readings. The description is verbose on purpose, and the prompt insists on verbatim transcription where possible.

Embedding

Embeddings come from ZeroEntropy's `zembed-1` via `src/embeddings.py`. The wrapper class `ZeroEntropyEmbeddings` follows LangChain's Embeddings contract. It exposes `embed_documents` for the indexer and `embed_query` for the retrieval node.

The important detail is the `input_type` parameter. ZeroEntropy is asymmetric. Document embeddings and query embeddings are produced in different modes so the model can specialise its representation for each role. The wrapper handles that automatically based on which method you call. A 2560 dimensional vector is returned in either case.

```
def embed_documents(self, texts):
    return self._embed(texts, input_type="document")

def embed_query(self, text):
    return self._embed([text], input_type="query")[0]
```

The single collection design

One ChromaDB collection (property_law_kb) holds everything. Text and slide chunks live side by side. Metadata filters distinguish them. This is intentional. It keeps retrieval simple. Filter rules build dictionaries that ChromaDB applies natively, so the same MMR call covers both modalities or just one.

The filter builder, with the three cases it handles

```
def _build_filter(week=None, doc_types=None):
    conditions = []
    if week:
        conditions.append({"week": {"$eq": week}})
    if doc_types and len(doc_types) == 1:
        conditions.append({"type": {"$eq": doc_types[0]}})
    elif doc_types and len(doc_types) > 1:
        conditions.append({"type": {"$in": doc_types}})
    if not conditions:
        return None
    if len(conditions) == 1:
        return conditions[0]
    return {"$and": conditions}
```

Three return shapes. None when no constraints were given, a single condition, or an explicit \$and when both week and type need to apply.

Layered Retrieval Pipeline

Retrieval in ashGPT is not a single similarity search. It is four layered stages, each responsible for a different failure mode. The whole stack lives in src/agent/tools.py with the cross encoder isolated in src/agent/reranker.py. The retrieval node calls retrieve_all, which fans out to retrieve_texts and retrieve_slides. Both follow the same four stage pattern with different doc type filters.

flowchart LR Q["Query (packed if follow up)"] --> EMB["zembed-1 query

mode"] EMB --> MMR["MMR. fetch_k=20, lambda=0.5

Returns 16 text or 8 slide candidates"] MMR --> META["Metadata filter

week and doc_type at query time"] META --> RR["Cross encoder reranker

ms-marco-MiniLM-L-6-v2"] RR --> OUT["Top 8 text + top 4 slide chunks"]

Stage 1. Asymmetric query embedding

The user's query is embedded with zembed-1 in query mode. Because the embedder is asymmetric, the resulting vector is tuned for "short question" representation. A document chunk indexed earlier was embedded in document mode, tuned for "long passage". The dot product between query mode and document mode is what the model is trained on, which gives more meaningful retrieval than a symmetric embedder would for the same content.

Stage 2. Maximal Marginal Relevance over fetch

Plain similarity returns near duplicates. Legal text has a lot of those. The same paragraph from a reading might appear three times because two tutorials and a slide all paraphrase it. MMR re ranks the top fetch_k candidates so that each new pick is rewarded for being relevant and penalised for being too similar to picks already in the result. Lambda 0.5 splits the trade off evenly.

Crucially, MMR is configured to *over fetch*. It does not return the top eight directly. It returns sixteen text candidates and eight slide candidates. The over fetch creates a diverse candidate pool that the cross encoder can then re score in stage four.

```
def _search(store, query, k, where_filter, strategy=None):
    strat = strategy or RETRIEVAL_STRATEGY # "mmr"
    if strat == "mmr":
        return store.max_marginal_relevance_search(
            query, k=k, fetch_k=MMR_FETCH_K,
            lambda_mult=MMR_LAMBDA, filter=where_filter,
        )
    return store.similarity_search(query, k=k, filter=where_filter)
```

Stage 3. Metadata filtering

The router can pull a week filter out of the user's wording. The Streamlit sidebar can also pre set one, and that overrides whatever the model thinks. The filter is applied at query time as a ChromaDB where clause. The doc type filter is hardcoded inside `retrieve_texts` (reading, tutorial, supplementary) and `retrieve_slides` (lecture_slide). That separation is what lets the agent pull both modalities into different state slots.

Stage 4. Cross encoder reranker

The cross encoder is a small pretrained model (ms-marco-MiniLM-L-6-v2) that scores each query and chunk pair directly, rather than relying on the dot product of pre computed embeddings. It is roughly 80 MB on first download and runs locally. The retrieval helper passes the MMR pool into `get_reranker().rerank(query, docs, top_k=k)` which returns the same documents sorted by cross encoder score, truncated to the originally requested k.

Why both MMR and a cross encoder. MMR enforces diversity, which the IRAC and chronology nodes need. A cross encoder ranks pure relevance, which lifts the most useful chunk to the top. Doing them in sequence gets the best of both. The diversity comes first, so the candidate pool spans different sources. The relevance comes second, so the top k inside that diverse pool is whatever scores highest against the actual question.

```
def _maybe_rerank(query, docs, top_k, use_reranker):
    if not use_reranker:
        return docs[:top_k]
    try:
        from src.agent.reranker import get_reranker
        reranker = get_reranker()
        return reranker.rerank(query, docs, top_k=top_k)
    except Exception as e:
        log.warning("Reranker unavailable (%s); falling back to MMR order", e)
        return docs[:top_k]
```

The eval harness can disable the reranker by passing `use_reranker=False` through the agent state. That is exactly how the no reranker ablation is implemented. Same MMR pool, no re scoring, the original MMR order is truncated to top eight or top four directly.

Example. `retrieve_all` on the Moran query

Call

`retrieve_all("What is the legal test from Buckinghamshire County Council v`

`Moran?", week=None, k_text=8, k_slides=4, use_reranker=None)`

Step 1

retrieve_texts applies the doc type filter [reading, tutorial, supplementary].

With USE_RERANKER=True it asks for RERANKER_FETCH_K_TEXT = 16 from

_search.

Step 2

MMR over the Chroma collection returns 16 diverse text candidates.

_raw_to_retrieved converts each LangChain Document to a

RetrievedDocument dict.

Step 3

_maybe_rerank calls get_reranker().rerank(query, 16 docs, top_k=8). The

cross encoder predicts a score for each (query, chunk) pair and the helper

sorts and truncates.

Step 4

Same flow for slides with `RERANKER_FETCH_K_SLIDES = 8` and `top_k=4`.

Returns

A tuple of two lists. Eight text dicts and four slide dicts. Each carries content,

source, week, doc_type, and image_path for slides.

Code link. `retrieve_texts` reads `USE_RERANKER` from config when no

override is given. With reranker on, `fetch_k` is 16 for texts and 8 for slides.

With it off, `fetch_k` is the requested `k` directly. The reranker call is wrapped

in a try except so a missing model file never crashes retrieval.

The packed retrieval query for follow ups

On a follow up turn the new student message alone is often too short to retrieve anything useful. `build_retrieval_query` in `chat_memory.py` prepends a short excerpt of the previous tutor reply, so the embedding picks up the topic the student is referring to. The trick is keeping that excerpt short. Dumping a full prior IRAC into the embedding washes out the new question. The cap is `CHAT_HISTORY_MAX_ASSISTANT_TAIL_CHARS`, set to 1200 characters in `config.py`.

The Cross Encoder Reranker

`src/agent/reranker.py` wraps a small pretrained cross encoder. Two parts. A module level singleton accessor and the wrapper class itself.

The lazy singleton

Loading a sentence transformer model takes a few seconds and consumes about 80 MB of memory once it is on disk. The agent does not need that cost on import. `get_reranker` creates the wrapper instance the first time it is called and caches it in a module level variable. Every subsequent call reuses it.

```
_singleton = None

def get_reranker():
    global _singleton
    if _singleton is None:
        _singleton = CrossEncoderReranker(model_name=RERANKER_MODEL)
    return _singleton
```

The wrapper class

The class itself defers the heavy work even further. The model is only built on the first call to rerank. Importing sentence_transformers lives inside _ensure_loaded so the import cost is also lazy. If the package is missing, the helper raises a clear ImportError that names the install command.

```
class CrossEncoderReranker:
    def __init__(self, model_name=RERANKER_MODEL):
        self.model_name = model_name
        self._model = None

    def _ensure_loaded(self):
        if self._model is None:
            try:
                from sentence_transformers import CrossEncoder
            except ImportError as e:
                raise ImportError(
                    "sentence-transformers is required for CrossEncoderReranker. "
                    "Install it with `pip install sentence-transformers>=3.0.0`."
                ) from e
            log.info("Loading cross-encoder reranker: %s", self.model_name)
            self._model = CrossEncoder(self.model_name)
```

The rerank call

The actual scoring is a single call to self._model.predict. The input is a list of (query, content) string pairs. The output is a list of float scores aligned with the input pairs. The helper zips documents with scores, sorts descending, and truncates to top_k. Empty input or zero top_k short circuits to an empty list.

```
def rerank(self, query, documents, top_k):
    if not documents:
        return []
    if top_k <= 0:
        return []
    self._ensure_loaded()
    pairs = [(query, doc.get("content", "")) for doc in documents]
```

```
try:
    scores = self._model.predict(pairs)
except Exception as e:
    log.warning("Reranker failed (%s); returning original order", e)
    return documents[:top_k]
scored = list(zip(documents, scores))
scored.sort(key=lambda x: float(x[1]), reverse=True)
return [doc for doc, _ in scored[:top_k]]
```

Example. Reranker on a five candidate pool

Call

`reranker.rerank("ratio in Moran", docs=[d0,d1,d2,d3,d4], top_k=3)` where d2

has Moran ratio language verbatim, d0 is paraphrase, d1 is unrelated.

Step 1

Pairs become `[("ratio in Moran", d0.content), ("ratio in Moran", d1.content),`

`("ratio in Moran", d2.content), ...]`.

Step 2

predict returns scores like [3.2, -1.4, 6.7, 0.9, -0.2]. Higher is more relevant.

Returns

[d2, d0, d3]. d2 first because the verbatim Moran ratio scores highest. d0

second because paraphrase scores positive. d3 third. d1 and d4 truncated.

Code link. The defensive try except mirrors the shape used in llm.py. If

predict raises for any reason, retrieval still returns the first top_k documents

in MMR order. Failure mode degrades to "agent without reranker", never to "no

retrieval".

Why a cross encoder rather than a bi encoder

A bi encoder embeds the query and the document independently and compares with a dot product. ZeroEntropy is a bi encoder, so the initial Chroma search is bi encoder ranking. A cross encoder takes the query and the document together as a single input and produces a relevance score. That direct interaction is more accurate than dot product comparison but cannot scale to millions of documents because it requires one forward pass per pair. The hybrid pattern here is exactly the right shape. Use the bi encoder to fetch a small candidate pool with MMR diversity, then use the cross encoder to re score that small pool. Twenty four pairs per query for retrieval, eight for slides, well within budget for a study tool.

Reranker contribution from the eval suite. Average context precision rises from 0.59 without the reranker to 0.66 with it. The reranker improves precision on 11 of 22 queries, hurts precision on 3 queries, and ties on 8 queries. MRR moves from 0.93 to 0.97. NDCG at K is essentially flat at 0.94 either way. The principal effect is precision, not ranking order, which is consistent with the design role of the layer.

The State Schema

LangGraph runs on top of a typed state object. ashGPT defines that state in `src/agent/state.py` as a TypedDict with `total=False`. Every key is optional at graph entry. Nodes return partial dicts and LangGraph merges them into the running state.

WHAT FLOWS THROUGH THE GRAPH

Phase	Key	Type	Written by
Input	query	str	Caller (Streamlit or eval harness)

Phase	Key	Type	Written by
	week_filter	str None	Caller, possibly overwritten by Router
	chat_history	list[ChatMessage]	Caller. Prior turns only.
	use_reranker	bool None	Eval harness only. Forces reranker on or off for ablation runs.
Router	intent	"ratio" "chronology" "summary" "general"	Router node
Retrieval	retrieved_texts	list[RetrievedDocument]	Retrieval node
	retrieved_slides	list[RetrievedDocument]	Retrieval node
Ratio	ratio_decidendi	str	Ratio Extractor
	irac_analysis	str	Ratio Extractor
Chronology	mermaid_diagram	str	Chronology Generator
	chronology_summary	str	Chronology Generator
Synthesis	final_answer	str	Synthesis (and possibly rewritten by Verification)
Verification	verification_report	{"unsupported_claims": list[str],	Verification node

Phase	Key	Type	Written by
		"rewrites_applied": bool}	
Diagnostics	node_trace	list[str]	Every node appends

Why total equals false

Total false means every key is optional. A new state can be passed in with just a query and the graph still runs. Every node only writes the keys it owns. This pattern keeps node code tight. A node returns {"intent": "ratio", "node_trace": [...]} rather than rebuilding the whole state.

The reranker override field

use_reranker is a state level escape hatch. It defaults to None, in which case retrieval falls back to the global USE_RERANKER from config. The eval harness sets it to False for the no reranker ablation. This is what lets the harness re run the full graph for the same case with the cross encoder disabled, without touching config.

RetrievedDocument

```
class RetrievedDocument(TypedDict):
    content: str
    source: str
    week: str
    doc_type: str
    image_path: str | None
```

Example. A text chunk vs a slide chunk

Text chunk

```
{"content": "Adverse possession requires...", "source": "Sackville_Ch2.pdf",
```

```
"week": "week_3", "doc_type": "reading", "image_path": null}
```

Slide chunk

```
{"content": "Heading. Possession of Chattels...", "source": "slide_05.jpg",
```

```
"week": "week_3", "doc_type": "lecture_slide", "image_path":
```

```
"data/week_3/lecture/slide_05.jpg"}
```

Code link. The shape is identical. Only doc_type and image_path differ.

Downstream nodes treat them differently because of doc_type.

The verification report

After synthesis the verification node attaches a small report dict to state. It always contains a list of unsupported claims (empty when everything checks out) and a boolean indicating whether a rewrite was applied. The eval harness reads this dict for live telemetry during a run and the failure analysis can quote it.

```
verification_report = {  
    "unsupported_claims": ["Smith v Jones"], # citations missing from sources  
    "rewrites_applied": True,               # synthesis was asked to rewrite
```

```
}
```

The LangGraph State Machine

The graph itself lives in `src/agent/graph.py`. The construction is short, but every line matters because of how LangGraph compiles edges.

Wiring the graph

```
graph = StateGraph(AgentState)

graph.add_node("router", router_node)
graph.add_node("retrieval", retrieval_node)
graph.add_node("ratio_extractor", ratio_extractor_node)
graph.add_node("chronology", chronology_node)
graph.add_node("synthesis", synthesis_node)
if USE_VERIFICATION:
    graph.add_node("verification", verification_node)

graph.set_entry_point("router")
graph.add_edge("router", "retrieval")

graph.add_conditional_edges(
    source="retrieval",
    path=_route_after_retrieval,
    path_map={
        "ratio": "ratio_extractor",
        "chronology": "chronology",
        "summary": "ratio_extractor",
        "general": "synthesis",
    },
)

graph.add_conditional_edges(
    source="ratio_extractor",
    path=lambda state: "chronology" if state.get("intent") == "summary" else "synthesis",
```

```

    path_map={"chronology": "chronology", "synthesis": "synthesis"},
)

graph.add_edge("chronology", "synthesis")

if USE_VERIFICATION:
    graph.add_edge("synthesis", "verification")
    graph.add_edge("verification", END)
else:
    graph.add_edge("synthesis", END)

```

How the conditional edges work

LangGraph supports two kinds of edges. A static edge always points to the same next node. A conditional edge invokes a small Python function that reads the current state and returns the name of the next node. ashGPT uses two conditional edges. The first picks the path after retrieval based on the router's intent. The second handles the chained case where summary needs both reasoning nodes.

Static edges look obvious but have a subtle role. Each one is a contract. chronology -> synthesis is always taken. synthesis -> verification is always taken when verification is enabled. verification -> END closes every path. The mix of static and conditional edges keeps the graph readable.

The four routes

Intent	Path	Outputs produced
ratio	router → retrieval → ratio_extractor → synthesis → verification	IRAC, ratio decidendi, final answer (verified).
chronology	router → retrieval → chronology → synthesis → verification	Mermaid diagram, timeline summary, final answer (verified).

Intent	Path	Outputs produced
summary	router → retrieval → ratio_extractor → chronology → synthesis → verification	IRAC, ratio decidendi, Mermaid diagram, timeline summary, final answer (verified).
general	router → retrieval → synthesis → verification	Final answer (verified). No specialised reasoning.

```
flowchart TD
ROUTE[router] --> RET[retrieval]
RET --> COND1{intent}
COND1
```

```
-- ratio --> RX[ratio_extractor]
COND1 -- chronology --> CG[chronology]
```

```
COND1 -- summary --> RX
COND1 -- general --> SYN[synthesis]
RX -->
```

```
COND2{intent equals summary}
COND2 -- yes --> CG
COND2 -- no --> SYN
```

```
CG --> SYN
SYN --> VER[verification]
VER --> END([END])
```

The cached singleton

Compiling a LangGraph is not free. It builds an internal representation, validates the edges, and hashes the node functions. The repository compiles once and caches via `get_graph`. `run_query` calls that cache. This means the first message of a session pays the compile cost and every subsequent message is fast.

```
_compiled_graph = None
```

```
def get_graph():
```

```
global _compiled_graph
if _compiled_graph is None:
    _compiled_graph = build_graph()
return _compiled_graph
```

The public entry point

```
def run_query(query, week_filter=None, chat_history=None):
    graph = get_graph()
    initial_state = {"query": query}
    if week_filter:
        initial_state["week_filter"] = week_filter
    prepared = prepare_chat_history_for_run(chat_history)
    if prepared:
        initial_state["chat_history"] = prepared
    return graph.invoke(initial_state)
```

Example. End to end run_query trace with verification

Input

```
run_query("What is the ratio in Perry v Clissold?", week_filter=None,
```

```
chat_history=None)
```

Step 1

Initial state becomes {"query": "What is the ratio in Perry v Clissold?"}. No

week, no history.

Step 2

graph.invoke runs router, retrieval, ratio_extractor, synthesis, verification.

Each node merges its partial dict into the running state.

Returns

A dict with all keys populated. intent="ratio", retrieved_texts with 8 chunks,

retrieved_slides with 4 chunks, ratio_decidendi one sentence, irac_analysis

full IRAC, final_answer the tutor reply,

verification_report={"unsupported_claims":[],"rewrites_applied":False},

node_trace=["router","retrieval","ratio_extractor","synthesis","verification"].

Code link. The cached `get_graph` avoids recompiling on every call. The

merge of partial dicts into the running state is handled by LangGraph itself, the

agent only writes the keys it owns. Verification is the final node and may

rewrite `final_answer` in place if any citation is unsupported.

`run_query` is the only function the rest of the system needs to call. It builds the initial state, sanitises any chat history, invokes the graph, and returns the merged final state. Streamlit calls it. The eval harness calls it (or its per node timed equivalent `run_agent_with_metrics`). The tests call it.

The Six Cognitive Nodes

Every node in `src/agent/nodes.py` is a pure function. State in. Partial state out. The router classifies. Retrieval reads from Chroma and runs the layered pipeline. Ratio extractor and chronology generator each do one specialised reasoning task. Synthesis grounds the final answer. Verification fact checks every citation in the final answer. The same shape repeats six times.

1. Router 2. Retrieval 3. Ratio Extractor 4. Chronology 5. Synthesis 6.

Verification

Router intent classifier and week filter extractor

Reads. `query`, `chat_history`, an optional pre set `week_filter`. **Writes.** `intent`, `week_filter`, `node_trace`.

The router is the most important node for hypothesis testing. If routing is wrong, ratio cases never run the ratio path and chronology cases never get a diagram. Its system prompt is therefore very explicit about the four classes and the decision rule that disambiguates summary from chronology.

The router model is gemini-3.1-flash-lite-preview. It is the cheapest model in the pipeline because the task is just classification. The output is parsed by stripping any optional code fences and calling `json.loads`. If parsing fails the router defaults to summary so the user still gets a complete answer.

WEEK FILTER PRECEDENCE

If the Streamlit sidebar already passed a `week_filter`, that value wins. The router only sets `week_filter` when the caller did not. This gives the user explicit control.

```
existing_week = state.get("week_filter")
week_filter = existing_week if existing_week else parsed_week
```

WHY JSON OUTPUT

Free form classification is brittle. Forcing JSON pins the contract. The same parser shape is reused for the judges later, where it matters even more. If a future model adds Markdown fences around the JSON, the regex strip handles that case.

Example. Router across all four intents

Input A

```
state["query"] = "What is the ratio decidendi in Moran?"
```

LLM raw

```
{"intent": "ratio", "week_filter": null}
```

Input B

```
state["query"] = "Show me a flowchart of Perry v Clissold."
```

LLM raw

```
{"intent": "chronology", "week_filter": null}
```

Input C

```
state["query"] = "Walk me through Moran and give me a summary"
```

LLM raw

`{"intent": "summary", "week_filter": null}`. The decision rule fires because

timeline language sits next to summary language.

Input D

`state["query"] = "Summarise the week 3 readings on adverse possession"`

LLM raw

`{"intent": "summary", "week_filter": "week_3"}`. The router pulls the week

filter out of the wording.

Code link. The `re.sub(r"```json\s*|\s*```", "", raw)` strip is what tolerates

fenced JSON. Then `json.loads`. Then the intent is re validated against the

allowed set. Failure routes to summary as the safe default.

Retrieval the senses of the agent

Reads. query, week_filter, chat_history, optional use_reranker. **Writes.** retrieved_texts, retrieved_slides, node_trace.

Retrieval is the only node that talks to ChromaDB. It does so through retrieve_all in src/agent/tools.py. The function makes two calls. One restricted to text types (reading, tutorial, supplementary). One restricted to lecture_slide. The two halves of the result remain in separate state slots, which is what allows downstream prompts to label sources distinctly.

```
def retrieval_node(state):
    history = get_chat_history(state)
    search_q = build_retrieval_query(state["query"], history)
    week = state.get("week_filter")
    use_reranker_override = state.get("use_reranker")
    texts, slides = retrieve_all(
        search_q,
        week=week,
        k_text=8,
        k_slides=4,
        use_reranker=use_reranker_override,
    )
    return {
        "retrieved_texts": texts,
        "retrieved_slides": slides,
        "node_trace": _append_trace(state, "retrieval"),
    }
```

The four stage layered pipeline (Section 5) executes inside retrieve_texts and retrieve_slides. The retrieval node itself is small. It packs the query for follow ups, hands off to the helpers, and writes the result into state.

Example. Retrieval against a real query

State in

```
{"query": "What is adverse possession?", "intent": "ratio", "week_filter": null,
```

```
"node_trace": ["router"]}
```

Step 1

```
build_retrieval_query("What is adverse possession?", []) returns the query
```

unchanged because there is no chat history.

Step 2

retrieve_all issues two layered pipelines. retrieve_texts runs MMR over fetch

16, then cross encoder reranker reduces to 8. retrieve_slides runs MMR over

fetch 8, then reranker reduces to 4.

State delta

```
{"retrieved_texts": [...8 chunks...], "retrieved_slides": [...4 chunks...],
```

```
"node_trace": ["router", "retrieval"]}. Each chunk has content, source, week,
```

```
doc_type, and an image_path for slides only.
```

Code link. `k_text=8` and `k_slides=4` are hardcoded in the retrieval node.

`RERANKER_FETCH_K_TEXT=16` and `RERANKER_FETCH_K_SLIDES=8` in config

control the over fetch when the reranker is on. With `use_reranker=False` the

pipeline skips the cross encoder and truncates the MMR pool directly.

Ratio Extractor the IRAC engine

Reads. `query`, `retrieved_texts`, `retrieved_slides`, `chat_history`. **Writes.** `ratio_decidendi`, `irac_analysis`, `node_trace`.

This node implements one half of the design hypothesis. It separates the binding rule from the rest of the case material. Its system prompt is two paragraphs of constraint. Pull a single sentence ratio. Produce four labelled sections. Cite only what is in the source material.

Notice the careful split inside `RATIO_SYSTEM`. Facts are constrained. Explanations are encouraged. Without that distinction, the model either invents to fill gaps or refuses to clarify. The wording was chosen so a tutor style answer is allowed without inviting hallucination.

CAPTURING THE ONE SENTENCE RATIO

After the LLM responds, a small heuristic scans the output for the line that mentions both "ratio" and "decidendi". That line becomes ratio_decidendi. The full body becomes irac_analysis. The synthesis node consumes both later.

```
ratio_line = ""
for line in response.split("\n"):
    if "ratio" in line.lower() and "decidendi" in line.lower():
        ratio_line = line.strip().lstrip("#*- ")
        break
if not ratio_line:
    ratio_line = response.split("\n")[0].strip()
```

Example. Ratio extractor in action

State in

Has 8 text chunks and 4 slides loaded by retrieval. query="What is the ratio

in Buckinghamshire v Moran?"

LLM prompt

CURRENT USER QUESTION plus full SOURCE MATERIAL block. System

instruction is RATIO_SYSTEM.

LLM raw response

Ratio Decidendi: To establish adverse possession, the squatter must show

factual possession plus animus possidendi. 1. **Issue.** Whether the

defendant could establish adverse possession over the disputed land. 2.

Rule. A squatter must demonstrate factual possession (single, exclusive,

undisturbed control) and animus possidendi (the intention to possess as

owner). 3. **Application.** Moran enclosed and used the land continuously..."

Heuristic finds

The first line containing both "ratio" and "decidendi". After lstrip, ratio_line =

"Ratio Decidendi: To establish adverse possession..."

State delta

```
{"ratio_decidendi": "Ratio Decidendi: To establish...", "irac_analysis": "<full  
response>", "node_trace": [..., "ratio_extractor"]}
```

Code link. If no line matches, the fallback grabs the first non empty line of the response. This guarantees ratio_decidendi is never empty when irac_analysis is populated.

Chronology Generator the diagram engine

Reads. query, retrieved_texts, retrieved_slides, chat_history. **Writes.** mermaid_diagram, chronology_summary, node_trace.

This is the other half of the hypothesis. The diagram is the visible artefact that proves the agent reasoned about chronology as its own task. The system prompt insists on Mermaid graph TD, camelCase IDs, short labels, edge labels for detail, and 5 to 12 nodes.

PARSING THE RESPONSE

The node uses two regular expressions. One captures the fenced Mermaid block. The other captures the timeline summary that follows. Both are stored separately in state. Synthesis later inserts the Mermaid block verbatim into the final answer, and the timeline summary turns into prose.

```
mermaid_match = re.search(r"```mermaid\s*\n(.*)```", response, re.DOTALL)  
mermaid_diagram = mermaid_match.group(1).strip() if mermaid_match else ""
```

```
summary_match = re.search(
    r"\s*\s*Timeline Summary[:\s]*\s*(.*)",
    response,
    re.DOTALL | re.IGNORECASE,
)
chronology_summary = summary_match.group(1).strip() if summary_match else ""
```

WHY A SMALL MODEL IS FINE HERE

The chronology task is structurally constrained. There is no need for heavy reasoning. The default model is gemini-3-flash-preview. It is fast and cheap, and the system prompt does most of the structural work.

Synthesis the grounded final answer

Reads. Everything. **Writes.** final_answer, node_trace.

The synthesis node produces what the student actually reads. It receives the original query, the intent, the retrieved sources, and any artefacts from the upstream reasoning nodes. Its system prompt enforces three things. Ground every fact in the primary evidence. Treat IRAC and chronology as derived analysis that still needs verification. Reproduce any Mermaid block verbatim.

THE PRIMARY EVIDENCE VS DERIVED ANALYSIS SPLIT

The user prompt builds two clearly labelled blocks. PRIMARY EVIDENCE contains the raw retrieved chunks. DERIVED ANALYSIS contains the IRAC, the ratio sentence, the Mermaid block, and the timeline summary. The model is told to verify every claim from PRIMARY EVIDENCE, even if it appears in DERIVED ANALYSIS. This protects the system from a hallucination that snuck into ratio extraction propagating into the final answer.

WHY THE MERMAID BLOCK IS COPIED VERBATIM

If the synthesis model paraphrased the chart into bullet points, the Streamlit UI could not render it. The whole point of the chronology path would be lost in the last step. The system prompt and the user prompt both repeat the rule, and a follow up sentence makes it clear. *The UI renders the diagram, do not convert it to bullet points.*

The frontend reinforces this. If the synthesis output omits the Mermaid block but the state has one, the UI appends it before storing the message. That guardrail is what guarantees diagrams survive multi turn replay.

Verification citation regex and rewrite

Reads. final_answer, retrieved_texts, retrieved_slides, query. **Writes.** final_answer (possibly rewritten), verification_report, node_trace.

The verification node is the post synthesis safety net. It extracts every plausible X v Y case citation from the final answer, normalises each one, and checks for full string or two token presence in the concatenated retrieved chunk text. If any citation is unsupported, it asks the synthesis model to rewrite the answer with that claim removed or hedged, while preserving every other heading and any Mermaid block.

See the next section for the full anatomy of the citation extractor and the rewrite branch. The headline result from the eval suite is that the node fired on all 22 of 22 cases (the citation regex always runs) and the regex extracted zero unsupported citations across the suite, which is consistent with the synthesis prompt's strict grounding rules. The node remains active as a low cost safety net that intercepts hallucinated case citations the synthesis model might otherwise import from parametric memory.

The Verification Node

The verification node is split between two files. `src/agent/verification.py` contains the regex based citation extractor and a substring check against retrieved chunk text. `src/agent/nodes.py` contains the node itself, which orchestrates extraction, the unsupported list, and the rewrite branch.

Why a verification node at all

Synthesis is told strictly to ground every factual claim, but LLMs occasionally drift. A familiar Property Law case from the model's parametric memory might surface in the final answer even if the retrieval pool does not contain that case. The verification node catches that drift cheaply. It does not call any LLM by default. It only invokes the synthesis model if a citation needs to be removed or hedged.

The citation regex

The extractor is intentionally permissive on formatting (italics, underscores, plain text) but constrained on shape (X v Y or X vs Y). It also tolerates multi word case names like Buckinghamshire County Council v Moran by allowing up to four extra capitalised tokens on the left side, with stop words like "of", "the", "and" permitted between them.

```
_CASE_NAME_PART = (
    r"[A-Z][A-Za-z'-]+"
    r"(?:\s+(?:of|the|and|de|du|von|van)\s+[A-Z][A-Za-z'-]+)?"
    r"(?:\s+[A-Z][A-Za-z'-]+){0,4}"
)

_CASE_PATTERN = re.compile(
    r"(?P[*_] {1,2})?"
    rf"(?P{_CASE_NAME_PART})"
    r"\s+(?:v|vs)\.?\s+"
    rf"(?P{_CASE_NAME_PART})"
    r"(?P=wrap)?",
    re.UNICODE,
```

)

Trimming sentence initial words

Without the trim, the multi word left side of the regex absorbs sentence initial English words like "In", "The", or "However". The extractor strips those leading stopwords before normalising the citation. The `_LEFT_TRIM_PREFIXES` set covers articles, prepositions, conjunctions, and a few legal connectors like "Section" and "See".

```
def _trim_leading_stopwords(left):
    parts = left.split()
    while parts and parts[0].lower() in _LEFT_TRIM_PREFIXES:
        parts.pop(0)
    return " ".join(parts)
```

Checking each citation against the sources

For each candidate citation the helper normalises the text (lowercase, strip italic markers, collapse whitespace) and asks two questions in order. Does the full X v Y string appear in the concatenated source text. If not, do both party tokens (the full left side and the full right side) appear individually. The two token fallback is what catches citations where the source quotes the case as "In Smith and Jones the court held..." without the v separator.

```
def case_appears_in_sources(case, sources_text):
    norm_sources = _normalise(sources_text)
    norm_case = _normalise(case)
    if norm_case in norm_sources:
        return True
    parts = [p.strip() for p in re.split(r"\sv\s", norm_case) if p.strip()]
    if len(parts) != 2:
        return False
    left, right = parts
    return (left in norm_sources) and (right in norm_sources)
```

The node itself

verification_node in nodes.py stitches the helpers together. It always logs into verification_report, regardless of whether the rewrite branch fires. If find_unsupported_cases returns an empty list, the node short circuits with the report and appends to the trace. If it returns one or more citations, the node builds a rewrite prompt and calls llm_call with the synthesis model and a dedicated rewrite system instruction.

```
def verification_node(state):
    answer = state.get("final_answer", "") or ""
    sources_text = _collect_sources_text(state)
    unsupported = find_unsupported_cases(answer, sources_text)

    report = {"unsupported_claims": unsupported, "rewrites_applied": False}

    if not unsupported:
        return {
            "verification_report": report,
            "node_trace": _append_trace(state, "verification"),
        }

    rewrite_prompt = (
        f"STUDENT QUESTION: {state.get('query', '')}\n\n"
        f"RETRIEVED SOURCE MATERIAL:\n{_format_context(state)}\n\n"
        f"DRAFT ANSWER:\n{answer}\n\n"
        f"UNSUPPORTED CITATIONS (must be removed or hedged):\n"
        + "\n".join(f"- {c}" for c in unsupported)
        + "\n\nRewrite the answer per the instructions. Return ONLY the rewritten answer."
    )

    try:
        rewritten = llm_call(
            rewrite_prompt,
            model=SYNTHESIS_MODEL,
            system_instruction=VERIFICATION_REWRITE_SYSTEM,
        )
        rewritten = (rewritten or "").strip()
        if rewritten:
            report["rewrites_applied"] = True
            return {
```

```

        "final_answer": rewritten,
        "verification_report": report,
        "node_trace": _append_trace(state, "verification"),
    }
except Exception as e:
    log.warning("VerificationNode: rewrite failed (%s); keeping draft", e)

return {
    "verification_report": report,
    "node_trace": _append_trace(state, "verification"),
}

```

The rewrite system instruction

The rewrite prompt tells the synthesis model exactly what to preserve and what to change. Every supported claim, every Markdown heading, every Mermaid block, must come through verbatim. Only the unsupported citations get removed or hedged. The model is also told not to introduce new citations and not to mention the rewrite process to the student.

```

VERIFICATION_REWRITE_SYSTEM = (
    "You are a careful Property Law editor. You will be given:\n"
    "1. The student's question.\n"
    "2. The retrieved source material.\n"
    "3. A draft answer.\n"
    "4. A list of CASE CITATIONS in the draft that DO NOT appear in the sources.\n\n"
    "Your task: rewrite the draft answer so that EACH unsupported citation is "
    "either (a) removed entirely, or (b) hedged with a phrase that signals it is "
    "not in the provided materials (e.g. \"is not covered in the indexed "
    "sources\"). Preserve every supported claim, the structure of the answer, "
    "every Markdown heading, and any ```mermaid code block VERBATIM. Do not "
    "introduce new citations. Do not mention this rewrite process to the "
    "student. Return ONLY the rewritten answer."
)

```

Example. Verification on a clean answer (no rewrite)

final_answer

"...the court in Buckinghamshire County Council v Moran held that..."

Extracted citations

["Buckinghamshire County Council v Moran"]

Sources contain

"Buckinghamshire" appears in two slide chunks and "Moran" appears in three

reading chunks. The full string also appears in slide_05 caption.

Report

```
{"unsupported_claims": [], "rewrites_applied": False}. Rewrite branch never
```

fires.

Code link. The regex check costs sub millisecond. The eval suite shows the

node averaging 0.0 seconds latency because the rewrite branch did not fire

across all 22 cases.

Example. Verification on a hallucinated citation (rewrite fires)

final_answer

"...the principle from Smith v Jones that a squatter must show animus..."

Extracted citations

["Smith v Jones"]

Sources contain

No "Smith" anywhere. No "Jones" anywhere. Full string check fails. Two token

fallback also fails.

Rewrite branch

Synthesis model called with VERIFICATION_REWRITE_SYSTEM and the

unsupported list. Returns the same answer minus the Smith v Jones reference,

with all other content intact.

Report

```
{"unsupported_claims": ["Smith v Jones"], "rewrites_applied": True}.
```

final_answer is now the rewritten string.

Code link. If the rewrite call raises (rate limit, timeout) the node logs the

warning and keeps the original draft answer. The report still records the

unsupported citations so the eval harness can flag the case.

What the eval harness measures

A small live counter inside `run_evals.py` records, per case, whether the node fired, whether a rewrite was applied, and the cumulative count of unsupported citations seen. The log line looks like `[verification] case 7/22 fired=7 rewrites=2 unsupported_total=3`. Across the full 22 case suite the regex extracted zero unsupported citations and no rewrite branch was triggered. The node still ran on every case so its presence is recorded in the trace.

Why "fired on all 22 cases" is true even with zero rewrites. The verification node always runs the citation regex against the final answer. If no unsupported citation is found, it short circuits with an empty report. "Fired" means the node ran. "Rewrite applied" means the rewrite branch was triggered. Across the 22 case suite, fired count is 22 of 22, rewrite count is 0 of 22.

Multi Provider LLM Layer

One module, `src/llm.py`, hides the differences between Google, OpenAI, and Anthropic. Every node calls a single function, `llm_call(prompt, model, system_instruction, temperature)`. The dispatcher reads the model name prefix and forwards to the right SDK.

```
def llm_call(prompt, model, system_instruction=None, temperature=0.2, reasoning_effort=None):
    if model.startswith("gemini"):
        return _call_gemini(prompt, model, system_instruction, temperature)
    elif model.startswith("gpt"):
        effort = reasoning_effort or _REASONING_EFFORT.get(model)
        return _call_openai(prompt, model, system_instruction, temperature, effort)
    elif model.startswith("claude"):
        return _call_anthropic(prompt, model, system_instruction, temperature)
    else:
        raise ValueError(f"Unknown model prefix: {model!r}")
```

Lazy clients

The Gemini, OpenAI, and Anthropic SDK clients are only instantiated when first needed. If a configuration uses only Gemini and OpenAI, the Anthropic SDK is never imported. This keeps cold start fast and the dependency footprint minimal during runtime even though the requirements file installs all three.

Per provider quirks

Provider Special handling

Gemini	Calls <code>generate_content</code> with a <code>GenerateContentConfig</code> that includes the system instruction. Token usage comes from <code>usage_metadata</code> .
OpenAI	Uses the new <code>responses.create</code> API. System instructions are passed as the first message. Reasoning models accept a <code>reasoning_effort</code> hint, which is auto stripped on retry if the model rejects it.
Anthropic	Standard <code>messages.create</code> . <code>max_tokens</code> is hard coded to 4096. System prompt sits in the top level system field.

Token tracking

Every successful call records input and output token counts on a global `TokenUsage` object. Nodes never need to think about tracking. The eval harness calls `reset_token_usage` before each case and `get_token_usage().summary()` after, which is how the per case totals end up in the eval JSON.

Why temperature equals 0.2 by default. Reasoning is meant to be deterministic enough for a study aid, but slight diversity helps the model produce natural prose. Judges always use temperature 0 because they should not vary across replays.

Multi Turn Chat Memory

src/agent/chat_memory.py handles three concerns. Sanitising raw history. Formatting history for LLM prompts. Building a focused retrieval query when the user is following up.

Sanitising history

```
def prepare_chat_history_for_run(raw):
    if not raw:
        return []
    out = []
    for m in raw:
        role = m.get("role", "")
        content = (m.get("content") or "").strip()
        if role not in ("user", "assistant") or not content:
            continue
        cap = CHAT_HISTORY_MAX_CHARS_PER_MESSAGE # 3500
        if len(content) > cap:
            content = content[:cap] + "\n... [truncated]"
        out.append({"role": role, "content": content})
    if len(out) > CHAT_HISTORY_MAX_MESSAGES: # 24
        out = out[-CHAT_HISTORY_MAX_MESSAGES:]
    return out
```

Empty messages and unknown roles are dropped. Any single message longer than 3500 characters is hard truncated. The whole history is capped at the most recent 24 messages. These limits live in config.py so they can be tuned without code changes.

Formatting history

For LLM prompts the history is rendered as a numbered transcript with the labels Student and Tutor. This is what the router, ratio extractor, chronology generator, and synthesis node all see when there are prior turns.

```
def format_transcript_for_llm(messages):
    parts = []
    for i, m in enumerate(messages, start=1):
        label = "Student" if m["role"] == "user" else "Tutor"
        parts.append(f"[{i}] {label}: {m['content']}")
    return "\n\n".join(parts)
```

Packed retrieval query

This is the small detail that makes follow ups work without overwhelming the embedding. When the latest history entry is a tutor reply, a short excerpt of that reply is prepended to the new student question before embedding.

```
def build_retrieval_query(current_query, messages):
    if not messages:
        return current_query
    for i in range(len(messages) - 1, -1, -1):
        if messages[i]["role"] != "assistant":
            continue
        excerpt = messages[i]["content"][:CHAT_HISTORY_MAX_ASSISTANT_TAIL_CHARS].strip()
        if not excerpt:
            break
    return (
        "[Conversation follow-up — retrieve material relevant to both the prior "
        "answer and the new question.]\n\n"
        f"Prior tutor answer (excerpt):\n{excerpt}\n\n"
        f"New student question:\n{current_query}"
    )
    return current_query
```

If the user asks "give me an exam tip tailored to that ratio" with no excerpt the embedding has nothing to anchor on. With a short excerpt the embedding moves toward the right region of the vector space and retrieval pulls back useful chunks.

Guided Examples

The next walkthroughs trace one query each through the entire pipeline. The traces show the input, what the router returns, what retrieval pulls, what each downstream node produces, what verification does, and what the user sees.

Example 1. Ratio path with verification

User input. "What is the ratio decidendi for adverse possession in Buckinghamshire County Council v Moran."

1

Router. Reads the query. Returns {"intent": "ratio", "week_filter": null}.

The query mentions "ratio decidendi" which is the trigger phrase.

2

Retrieval. Calls retrieve_all with no week filter. MMR over fetches 16 text

candidates and 8 slide candidates, then the cross encoder reranker truncates

to the top 8 text and 4 slide chunks against the original query.

3

Ratio extractor. Builds a prompt with the retrieved context. gpt-5.3-chat-

latest produces a structured IRAC. The first scan picks the one line ratio.

Both go into state.

4

Synthesis. Reads PRIMARY EVIDENCE plus DERIVED ANALYSIS. Produces a

clean tutor style explanation with inline citations like (Source: Readings

Week 3).

5

Verification. Extracts citations from the final answer. Finds

Buckinghamshire County Council v Moran. Checks against retrieved chunk

text. Both tokens present. No rewrite. Report attached with empty

unsupported list.

UI. Streamlit shows the trace banner, the IRAC under an expander, and the

source list at the bottom.

user What is the ratio decidendi for adverse possession in Moran.

```
intent => ratio
```

```
node_trace => ["router", "retrieval", "ratio_extractor", "synthesis",
```

```
"verification"]
```

```
verification_report => {"unsupported_claims": [], "rewrites_applied": false}
```

```
final_answer => "To establish adverse possession, the squatter must show
```

```
factual possession (single and exclusive control) plus animus possidendi
```

```
(intention to possess), as held in Buckinghamshire County Council v Moran."
```

Example 2. Chronology path

User input. "Show me a flowchart of the events in Perry v Clissold."

1

Router. Returns intent="chronology". The phrase "show me a flowchart"

with no request for explanation triggers the strict chronology branch.

2

Retrieval. Pulls chunks describing the dates of Clissold's enclosure, the

limitation period, and the title vesting. The reranker re scores against "events

in Perry v Clissold" and lifts the dated chunks above the doctrinal ones.

3

Chronology generator. Produces a Mermaid graph TD with five to seven

nodes, dates at the start of each label, and edge labels carrying the

connective detail.

4

Synthesis. Copies the Mermaid block verbatim, then writes a short

paragraph that introduces the diagram and ends with a one sentence

takeaway.

5

Verification. Extracts Perry v Clissold from the final answer. Confirms both

party names appear in the retrieved chunks. No rewrite.

graph TD e1["1881. Clissold takes possession"] -->|"encloses land, pays

rates"| e2["1881-1891. Continuous occupation"] e2 -->|"12 years pass"|

e3["1893. Limitation period expires"] e3 -->|"original owner's title

extinguished"| e4["Title vests in possessor"]

Example 3. Summary path

User input. "Walk me through Buckinghamshire v Moran and give me the sequence of events plus a summary."

1

Router. Detects both timeline language and explanation language. Returns

intent="summary". This is the path that fires both reasoning nodes.

2

Retrieval. Same MMR plus reranker pipeline. Eight text plus four slide

chunks loaded.

3

Ratio extractor. Produces the IRAC for Moran.

4

Chronology generator. Produces the Mermaid diagram of Moran's facts.

5

Synthesis. Combines the IRAC analysis, the diagram, and the source list

into a single tutor answer. The final reply contains a verbatim Mermaid block

plus the IRAC plus the explanatory prose.

6

Verification. Sweeps the final answer. Finds Moran citation. Confirmed. No

rewrite.

Example 4. General path

User input. "What is a fee simple."

1

Router. Returns intent="general". No need for IRAC, no need for a

diagram.

2

Retrieval. Pulls definition style chunks from readings and slides.

3

Synthesis. Goes straight from retrieval to a grounded plain prose

definition. node_trace is shorter. Latency is the lowest of all four paths (still

followed by verification).

4

Verification. No case citations to check. Empty report. Trace closes at

verification.

Example 5. Conversational follow up

Turn one. "Explain the Torrens system." **Turn two.** "How does that differ from a deeds registry."

On turn two the agent receives `chat_history` with both prior messages. The router uses the transcript to resolve "that" to "the Torrens system". The retrieval node calls `build_retrieval_query`, which prepends a short excerpt of the prior tutor answer. The MMR pool now reflects the topic. The reranker then re scores against the packed query. Synthesis stays grounded in the same indexed sources, and emits a one paragraph contrast. Verification confirms no unsupported citations.

The Streamlit Frontend

`app.py` is the chat interface. It is short on purpose. Most of the heavy lifting happens in the agent. The frontend's responsibilities are session memory, message rendering, and the trace banner.

Session memory

Streamlit's `st.session_state` holds a messages list. Each message is a dict with a role and content, exactly the shape the agent expects. When the user sends a new prompt, the prior messages become `chat_history` for that single `run_query` call.

render_message

This helper is the only place that decides how to display content. It splits text on the Mermaid fence regex. Plain segments go to `st.markdown`. Mermaid segments go to `st_mermaid`, which renders an interactive SVG diagram.

```
def render_message(content):
    parts = re.split(r"```mermaid\n.*?```", content, flags=re.DOTALL)
    for part in parts:
        m = re.fullmatch(r"```mermaid\n(.*)```", part, flags=re.DOTALL)
        if m:
            st_mermaid(m.group(1).strip(), height="auto")
        elif part.strip():
            st.markdown(part)
```

This same helper renders both the live response and any historic message during chat replay, which is exactly why diagrams keep working as the conversation grows.

Trace banner and expanders

After a response renders, three metric boxes appear above it. Detected intent. Total latency in seconds. The node trace as a single arrow joined string. Underneath, three optional expanders show the raw Mermaid code, the full IRAC analysis, and the source list. This is what makes the system inspectable while you study with it.

Sidebar

The sidebar exposes a week filter dropdown and a clear conversation button. Selecting a week from the dropdown sets `week_filter` on the next call and overrides anything the router would extract. Clear conversation wipes `st.session_state.messages` and reruns the page.

The Evaluation Suite

`src/eval/run_evals.py` turns the hypothesis into numbers. It runs four configurations on the same twenty two cases, scores them with LLM judges, and produces JSON, a Markdown failure analysis, and a directory of PNG plots.

Four configurations

FULL AGENT

The complete LangGraph pipeline. Every node fires according to intent.

Reranker on. Verification on. The full feature set under test.

PLAIN LLM BASELINE

One llm_call with BASELINE_MODEL and a tutor system instruction. No

retrieval, no graph, no IRAC, no diagram, no verification. The lower bound.

MEGA PROMPT ABLATION

Retrieval still runs (with reranker on). A single consolidated LLM call replaces

every reasoning node and the synthesis node. Measures whether multi node

decomposition adds anything beyond a well prompted single call.

AGENT WITHOUT RERANKER

Identical to the Full Agent except the cross encoder is disabled. MMR returns

8 text and 4 slide chunks directly. Isolates the reranker's contribution.

Query families

Every eval case is tagged with exactly one family so the report can show per family success and failure.

Family

What it stresses

factual_retrieval

Direct retrieval of doctrine, tests, and definitions that should appear in a small number of chunks.

cross_modal_retrieval

Questions phrased to target VLM described slides

Family	What it stresses
	alongside PDF readings. Confirms the multimodal index works.
analytical_synthesis	Multi evidence answers. Compare, relate, summarise across concepts, or reconstruct sequences using several sources.
conversational_followup	Two and three turn scripts. The second and third turns reference the first, and only the agent and the ablation see the prior turns. The baseline sees only the final turn.

What gets measured

Category	Metric	Notes
LLM judges	Groundedness 1 to 5	Two stage. Gemini drafts, GPT critiques. Final score is the critique value.
LLM judges	Answer relevancy 1 to 5	Single stage. Gemini judges only.
Retrieval	Precision at K, MRR, Per chunk binary relevance. Hit at K, NDCG at K	Computed for all configurations that retrieve.
Optional	Recall vs pool	With --retrieval-pool-eval, an enlarged retrieve and judge run. Bounded recall proxy, not corpus recall.
Structural	Mermaid validity	Five rule checks. Has graph header, has nodes, has edges, three node minimum, no empty labels.
Structural	IRAC compliance (intent conditional)	Looks for the four headings via case insensitive regex. Skipped (scored as not applicable) for chronology and general intents so the agent is not unfairly penalised on queries that should not have IRAC.

Category	Metric	Notes
Cost	Latency, per node	Captured per case. Per node latency is agent only. usage
	latency, token	
Cross modal	Both modalities retrieved	For each cross modal case, a flag records whether at least one slide and one text chunk came back.
Verification	Fired count, rewrite count, unsupported total	Live counter logged after each case. Tells the report whether the safety net intercepted any hallucinated citations.

The 22 cases

Six factual, five cross modal, seven analytical, and four conversational including a three turn case. The list is in EVAL_CASES. Each case carries an ID, a family tag, a rationale string, and one or more user turns.

The full case list

case id	family	turns
fact_moran_test	factual_retrieval	1
fact_fee_simple	factual_retrieval	1
fact_bailment_ratio	factual_retrieval	1
fact_perry_possessory	factual_retrieval	1
fact_fee_tail_vs_simple	factual_retrieval	1
fact_pla_chattels	factual_retrieval	1
xmodal_chattels_slides	cross_modal_retrieval	1
xmodal_moukataff_slide	cross_modal_retrieval	1
xmodal_ap_slides	cross_modal_retrieval	1

case id	family	turns
xmodal_torrens_indefeasibility	cross_modal_retrieval	1
xmodal_bailment_categories	cross_modal_retrieval	1
anal_ap_elements	analytical_synthesis	1
anal_lease_licence	analytical_synthesis	1
anal_chronology_ap	analytical_synthesis	1
anal_torrens_framework	analytical_synthesis	1
anal_taxonomy	analytical_synthesis	1
anal_legal_vs_equitable_interests	analytical_synthesis	1
anal_torrens_title_passing	analytical_synthesis	1
conv_ap_followup	conversational_followup	2
conv_torrens_followup	conversational_followup	2
conv_chattels_found	conversational_followup	2
conv_lease_three_turn	conversational_followup	3

Run order per case

1. Reset token tracker. Run the full agent (with reranker, with verification).
Record token usage.
2. Reset. Run the baseline against the final turn only. Record token usage.
3. Reset. Run the mega prompt ablation. Record token usage.
4. Reset. Run the agent again with use_reranker=False. Record token usage.
This is the no reranker ablation.
5. Score groundedness for all four using the appropriate retrieved context bundle.
6. Score relevancy for all four.

7. Score per chunk retrieval precision and ranking metrics for the agent, the mega prompt, and the no reranker ablation.
8. Score Mermaid validity and (intent conditional) IRAC compliance for all four.
9. If the case is cross modal, record the modality flags.
10. Update the live verification counter.

What lands on disk

eval_results.json Every case, every metric, every judge reasoning string, the verification report per case. eval_summary.json Aggregated metrics. Includes by_query_family and one entry per configuration including ablation_no_reranker. failure_analysis.md Per family checklist, low groundedness cases, agent vs ablation losses, retrieval misses, reranker contribution table. human_spot_check.md Five sampled cases with the LLM judge score and a hand filled human score for validation. *.png Plots covering groundedness, four way comparison, structural metrics, latency, source diversity, per node latency, multi metric, token usage, by family, reranker contribution.

LLM as Judge Framework

Three judges live inside the eval harness. Groundedness is the rich one. Relevancy and context precision are simpler.

Groundedness. Two stage cross provider

STAGE 1. DRAFT

Model. JUDGE_DRAFT_MODEL. Currently Gemini. Receives the question,

the source bundle, and the answer. Returns {"score": 1..5, "reasoning":

"..."}.

The rubric tells the judge to penalise invented facts but accept

paraphrase and plain English explanation.

STAGE 2. CRITIQUE

Model. JUDGE_CRITIQUE_MODEL. Currently OpenAI. Receives the same

materials plus the draft score and reasoning. Acts as a senior reviewer.

Returns {"score": 1..5, "reasoning": "...", "agreed_with_draft": bool}. The

critique score is the final number, with a fallback to the draft on parse failure.

Why two stages with two providers. To reduce same model self evaluation bias. If the agent is built mostly on OpenAI, a judge that is also OpenAI inherits the same blind spots. Cross provider judging breaks that loop. Stage two acts as a double check on stage one rather than as a fresh independent grader, which keeps cost and noise low.

Relevancy. Single stage

One call. Question and answer only, no source. The judge scores 1 to 5. Asks whether the answer addresses the question. This catches a high quality but off topic response that grounded scoring would miss.

Context precision. Per chunk binary

For each retrieved chunk the judge sees the question and the chunk content (truncated to 1500 characters), and returns {"relevant": true|false}. Then ranking metrics are computed from the verdict list in production order.

```
def _ranking_metrics_from_binary_verdicts(verdicts):
    k = len(verdicts)
    if k == 0:
        return {"mrr": 0.0, "hit_at_k": 0.0, "ndcg_at_k": 0.0, "k": 0}
    mrr = 0.0
    for i, v in enumerate(verdicts):
        if v:
            mrr = 1.0 / (i + 1); break
    hit_at_k = 1.0 if any(verdicts) else 0.0
    dcg = sum((1.0 if v else 0.0) / math.log2(i + 2) for i, v in enumerate(verdicts))
    rel_count = int(sum(1 for v in verdicts if v))
    idcg = sum(1.0 / math.log2(i + 2) for i in range(min(rel_count, k)))
    ndcg_at_k = (dcg / idcg) if idcg > 0 else 0.0
    return {"mrr": round(mrr, 3), "hit_at_k": hit_at_k,
            "ndcg_at_k": round(ndcg_at_k, 3), "k": k}
```

Optional pool eval

--retrieval-pool-eval retrieves a larger candidate pool (configured by EVAL_RETRIEVAL_POOL_K_*). Every chunk in the pool gets judged once. The production sized slice keeps its precision and ranking metrics. A new metric, recall vs pool, divides relevant in production by relevant anywhere in the pool. This is a bounded proxy for recall, since there are no human qrels for a corpus wide ground truth, and the report should treat it as such.

Structural checks

These do not call any model. They run rule based regular expressions.

- **Mermaid validity.** Five binary checks. Has graph header. Has nodes. Has edges. At least three nodes. No empty labels. Score is the fraction passed.
- **IRAC compliance.** Four binary checks for Issue, Rule, Application, Conclusion using case insensitive regex. The check is intent conditional. If the router classified the query as chronology or general, the helper returns {"applicable": False, "score": None} instead of zero. The summary excludes None scores from the average and reports irac_applicable_count alongside.

```
_IRAC_NA_INTENTS = {"chronology", "general"}

def check_irac_compliance(irac_text, intent=None):
    if intent in _IRAC_NA_INTENTS:
        return {"compliant": None, "score": None, "applicable": False, ...}
    if not irac_text or not irac_text.strip():
        return {"compliant": False, "score": 0.0, "applicable": True, ...}
    components = {
        "issue": bool(re.search(r"\b(issue|legal question)\b", irac_text, re.I)),
        "rule": bool(re.search(r"\b(rule|ratio decidendi|legal (rule|principle|test))\b", irac_text, re.I)),
        "application": bool(re.search(r"\bapplicat", irac_text, re.I)),
        "conclusion": bool(re.search(r"\bconclus", irac_text, re.I)),
    }
    ...
```

This is the conditional IRAC fix. Without it, the agent routed correctly to chronology for a "show me a flowchart" query and was penalised for not producing IRAC, even though IRAC was never asked for. The conditional check is fairer and is part of what the report explains.

Human Spot Check

The eval harness writes a small Markdown log at `eval_results/human_spot_check.md` with five deterministically sampled cases (seed 4205). Each entry shows the question, the agent's answer excerpt, and the LLM judge scores. A human is expected to fill in their own scores by hand, then re run the harness or compute the mean absolute error themselves.

The convention is that $|\text{human} - \text{judge}| \geq 2$ indicates a substantive rating gap that should be discussed in the report. Across the five sampled cases the human and judge agreed exactly on every case for both groundedness and relevancy. MAE 0.0. Five of five exact agreement. Zero substantive disagreements. This is qualitative validation that the LLM judge tracks human judgment for this dataset.

Metric	n	MAE	Exact agreement	Substantive disagreement
Groundedness	5	0.00	5/5	0/5
Answer relevancy	5	0.00	5/5	0/5

The five cases sampled were one factual retrieval (`fact_perry_possessory`), one factual retrieval definition (`fact_fee_simple`), one cross modal slide (`xmodal_moukataff_slide`), and two conversational follow ups (`conv_ap_followup`, `conv_chattels_found`). Sampling deterministically across families means the spot check is representative.

Why this matters. LLM as judge is only as credible as its agreement with human evaluators. With five of five exact agreement on a stratified sample, the LLM judge can be used to score the rest of the suite without each case needing a human grader.

Four Configurations and Ablations

The eval harness wires four configurations directly. Three are full LangGraph runs with different settings. One is a single LLM call with no graph at all.

Full Agent

The complete pipeline. Reranker on. Verification on. `run_agent_with_metrics` calls each node directly so per node latencies can be measured. Every node fires according to intent and the verification node runs after synthesis on every case.

Plain LLM Baseline

```
def run_baseline(query):
    start = time.time()
    answer = llm_call(
        query,
        model=BASELINE_MODEL,
        system_instruction="You are an Australian Property Law tutor. Answer the student's question as accurately as possible.",
    )
    elapsed = time.time() - start
    return {"answer": answer, "latency_s": round(elapsed, 2),
            "node_trace": ["baseline_llm"], "source_diversity": 0}
```

One LLM call. No retrieval. No graph. No history. `node_trace` is the literal string "baseline_llm". Source diversity zero because nothing is retrieved.

Mega Prompt Ablation

The ablation lives at the bottom of `nodes.py` as `mega_prompt_node`. It does not appear in the LangGraph graph at all. The eval harness wires it up directly when running the ablation configuration.

```
def run_ablation_mega_prompt(query, week=None, chat_history=None):
```

```
state = {"query": query, "intent": "summary"}
if week: state["week_filter"] = week
prepared = prepare_chat_history_for_run(chat_history)
if prepared: state["chat_history"] = prepared
state.update(retrieval_node(state))
state.update(mega_prompt_node(state))
...
```

Retrieval still runs (with the reranker). The ablation must see the same multimodal evidence the agent sees. After retrieval the mega prompt node fires once and is asked to produce all three sections in a single response. Section 1 is IRAC. Section 2 is a conditional Mermaid block. Section 3 is the plain English summary.

Parsing inside the ablation looks for the same artefacts the multi node graph would have produced. A ratio line. An IRAC block. A Mermaid block. A summary section. They are stored under the same state keys so the structural checks and judges treat them identically.

Agent without Reranker

Identical to the Full Agent except the eval harness passes `use_reranker=False` to `run_agent_with_metrics`. The retrieval helpers see the override and skip the cross encoder. MMR returns 8 text and 4 slide chunks directly. Verification still runs.

```
if len(case["user_turns"]) == 1:
    ablation_no_rr_result = run_agent_with_metrics(
        case["user_turns"][0], use_reranker=False
    )
else:
    ablation_no_rr_result = _run_agent_turns_for_eval(case, use_reranker=False)
```

The four way comparison in summary

	Plain LLM	Mega Prompt	Agent (no reranker)	Full Agent
Retrieval?	No	Yes (with reranker)	Yes (no reranker)	Yes (with reranker)
Graph?	No	No (single LLM call)	Yes (LangGraph)	Yes (LangGraph)
Reasoning nodes?	None	One mega prompt	All applicable	All applicable
Verification?	No	No	Yes	Yes
Isolates	Floor (parametric memory)	Cognitive separation claim	Reranker contribution	Full feature set

Real Evaluation Results

The numbers below come straight from `eval_results/eval_summary.json` after running `python -m src.eval.run_evals` on the twenty two case suite. They are the empirical answer to the design hypothesis.

Headline scoreboard

Four configurations, same twenty two cases, same retrieval bundle for groundedness scoring, same judges. Higher is better for everything except latency and tokens.

Metric	Plain LLM	Mega Prompt	Agent (no reranker)	Full Agent
Groundedness (1 to 5)	3.41	4.77	4.86	4.86

Metric	Plain LLM	Mega Prompt	Agent (no reranker)	Full Agent
Answer relevancy (1 to 5)	4.50	4.95	5.00	5.00
IRAC compliance (applicable cases)	0.10 (22)	1.00 (22)	1.00 (18)	1.00 (19)
Mermaid validity	0%	59%	68%	68%
Context precision @K	n/a	0.65	0.59	0.66
MRR	n/a	0.97	0.93	0.97
NDCG @K	n/a	0.94	0.94	0.94
Average end to end latency	3.72s	9.52s	22.11s	23.57s
Average tokens per query	357	10,320	27,204	26,144
Source diversity (unique sources)	0	6.05	6.05	5.91

What this proves. The full agent and the no reranker variant tie on groundedness at 4.86, both beating the mega prompt by 0.09 points on identical retrieved context. The reranker lifts context precision from 0.59 to 0.66, a seven point gain, while groundedness stays the same because synthesis is robust enough to filter noise either way. The plain baseline collapses to 3.41 with no retrieval, confirming the basic value of the indexed knowledge base.

The baseline is the floor

The Plain LLM Baseline scored 3.41 on groundedness, 0.10 on intent agnostic IRAC compliance, and zero on Mermaid validity. With no retrieval, the model fell back on what it learned during pretraining, which diverged from the LAWS2706 course materials. This is exactly why retrieval exists.

Attention dilution. The mega prompt failure mode

The mega prompt sees the same retrieved context as the full agent. Same evidence in. Different reasoning structure. Groundedness 4.77 versus 4.86. Mermaid validity 59% versus 68%. The mega prompt scored 1.00 IRAC compliance across all 22 cases, which sounds great until you see why. The consolidated prompt produces all three sections unconditionally, including for general greetings and pure chronology requests. This is over triggering. The full agent scored 1.00 IRAC compliance over its 19 applicable cases because the router correctly classified some queries as general or chronology and bypassed the ratio extractor by design.

Cognitive separation. The agent's advantage

The full agent achieved 4.86 on groundedness and a perfect 5.00 on relevancy. Groundedness was perfect on the cross modal and conversational families. The agent matches or beats the mega prompt on every individual case, and the failure analysis report confirms zero ablation wins.

The reranker contribution

The no reranker ablation isolates the cross encoder's contribution. With the reranker disabled, context precision drops from 0.66 to 0.59, MRR drops from 0.97 to 0.93, and NDCG stays flat at 0.94. The reranker improves precision on 11 of 22 queries, hurts precision on 3, and ties on 8. The principal effect is precision. Groundedness and relevancy stay tied because synthesis is robust to the swap, but the precision lift means the synthesis prompt sees fewer noise tokens, which keeps the latency cost stable while improving the quality of the evidence the model is asked to ground in.

Per family breakdown

Family	n	Groundedness	Relevancy	Context Precision
factual_retrieval	6	4.67 / 5	5.00 / 5	0.52

Family	n	Groundedness	Relevancy	Context Precision
--------	---	--------------	-----------	-------------------

cross_modal_retrieval	5	5.00 / 5	5.00 / 5	0.67
analytical_synthesis	7	4.86 / 5	5.00 / 5	0.74
conversational_followup	4	5.00 / 5	5.00 / 5	0.75

Cross modal retrieval hit both_modalities=True in 100% of cases. The dual index design works as intended. Conversational follow up reached the highest groundedness at 5.00 because the packed retrieval query plus the prior topic anchors the search well. Factual retrieval has the lowest precision at 0.52, with the Moran query and the Property Law Act 1974 chattels query each pulling only a small number of relevant chunks. Despite that, groundedness for factual was still 4.67. The synthesis node reliably filters noise even when retrieval is imprecise.

Latency breakdown

Node	Average latency	Why
verification	0.0s	Regex check is sub millisecond. Rewrite branch did not trigger across 22 cases.
router	1.1s	Cheapest model. Just a JSON classifier.
retrieval	2.7s	One ZeroEntropy embed plus two MMR searches plus the cross encoder rerank.
synthesis	4.6s	Strong model but a single grounded call.
ratio_extractor	6.6s	Mid tier model. IRAC structure plus ratio sentence.
chronology	8.6s	Slowest. Mermaid syntax plus timeline summary.

The 23.57s mean latency for the agent is dominated by chronology and ratio extractor when the route runs both. This is the cost of sequential node execution. The 0.09 point groundedness gain over the mega prompt corresponds directly to the reasoning labour that this sequential structure enables. Verification adds effectively zero because the rewrite branch never fired across the suite.

Token usage

Total tokens across all twenty two queries.

Baseline 7,864 tokens. 357 per query. Cheap and unreliable. Mega Prompt 227,045 tokens. 10,320 per query. One LLM call sees the whole bundle plus the consolidated directive. Agent (no reranker) 598,495 tokens. 27,204 per query. Slightly higher than the full agent because the noisier retrieval pool ends up in synthesis prompts more often. Full Agent 575,172 tokens. 26,144 per query. Higher than the ablations because the retrieved context is re presented to multiple nodes.

Verification telemetry

The live counter at the end of the run logged fired=22 rewrites=0 unsupported_total=0. The citation regex ran on every final answer. Zero unsupported citations were extracted across the suite. The rewrite branch never fired. This is the empirical answer to the question "does the verification node intercept anything in practice". The honest reading is that the synthesis grounding rules are strict enough that the safety net rarely catches anything, and the node remains active as a low cost guard rather than a frequent rewriter.

Systemic weaknesses

The agent's most consistent failure mode is retrieval imprecision, not reasoning error. Context precision averaged 0.66 with 15 of 22 queries falling under 0.70. This means the per node reasoning is robust enough to filter noise, but the cost is wasted context window tokens and elevated latency. Reasoning failures are rare. The factual statute lookup pattern, where a query like "what does the Property Law Act 1974 say about chattels" miss the surface form of the target, is the most consistent retrieval miss.

Future work signposted in the report

1. **Hybrid retrieval.** BM25 plus dense embeddings via reciprocal rank fusion would lift exact statute and case name surface forms above topical neighbours and address the 0.52 factual precision.
2. **Latency reduction.** Streaming node execution and smaller local models for the router and chronology could compress end to end latency toward 10 to 12 seconds while preserving routing accuracy.
3. **Broader verification.** The current verification node only checks case citations. Statute and slide identifier checks would extend the safety net to the families where the regex does not yet fire.

Verbatim Prompts

Every system prompt in the codebase is reproduced here so an interviewer can ask "show me the exact wording" and you can pull it up.

ROUTER_SYSTEM, in src/agent/nodes.py

```
ROUTER_SYSTEM = (  
    "You are a query classifier for a Property Law study assistant. "  
    "You may receive a prior conversation plus a LATEST student message. "  
    "Classify only the latest message, but use the transcript to resolve "  
    "vague references (e.g. \"that case\", \"the diagram\", \"explain further\").\n\n"
```

```

"Return ONLY a JSON object with two keys:\n"
' "intent": one of "ratio", "chronology", "summary", "general"\n'
' "week_filter": a string like "week_3" if the user mentions a specific '
"week, or null if not.\n\n"
"Intent definitions:\n"
'- "ratio": user wants the ratio decidendi, binding legal rule, or IRAC analysis of a case.\n'
'- "chronology": user ONLY wants a visual timeline, flowchart, diagram, chain of title, '
'sequence of events, chain of events, "who did what", "what happened and when", or '
'"walk me through the events" with NO request for a broader explanation or summary.\n'
'- "summary": use this when the user wants (a) a general case explanation OR topic overview, '
'OR (b) asks for BOTH a sequence/timeline AND a broader explanation or summary.\n'
'- "general": anything else (greetings, meta-questions, off-topic).\n\n'
"DECISION RULE: If the query contains timeline/sequence language alongside any "
"request for explanation, summary, or overview, classify as \"summary\", "
"NOT \"chronology\". Only use \"chronology\" when the user is asking solely for a "
"diagram or timeline with no broader explanation requested.\n\n"
"Respond with ONLY the JSON object, no markdown fences."
)

```

Why every clause is there. The first sentence pins the role. The conversation note enables follow up resolution. The JSON instruction pins the output contract. The four definitions teach the model to distinguish the three reasoning paths from each other and from the trivial general path. The decision rule was added because early routing tests misclassified summary requests as chronology when the wording leaned diagram heavy.

RATIO_SYSTEM

```

RATIO_SYSTEM = (
  "You are an expert Australian Property Law tutor. Your task is to analyse "
  "the provided source material and extract the ratio decidendi, the binding "
  "legal principle that the court applied to reach its decision.\n\n"
  "You MUST structure your response as an IRAC analysis:\n"
  "1. Issue: State the legal question the court had to decide.\n"
  "2. Rule: State the legal rule or principle (this IS the ratio decidendi).\n"
  "3. Application: Explain how the court applied the rule to the facts.\n"
  "4. Conclusion: State the court's decision.\n\n"
  "GROUNDING RULES:\n"
  "- FACTS must come from the sources: case names, dates, statutory references, "
  "holdings, and specific legal tests MUST appear in the source material. "
)

```

```
"Do NOT invent cases, statutes, or factual claims.\n"
"- EXPLANATIONS are encouraged: paraphrase, simplify, and explain.\n"
"- If you cite a case or statute by name, it MUST appear in the sources.\n"
"- If sources are insufficient, say so explicitly."
)
```

CHRONOLOGY_SYSTEM

```
CHRONOLOGY_SYSTEM = (
    "You are an expert Australian Property Law tutor specialising in creating "
    "precise chronological flowcharts of property transactions and legal events.\n\n"
    "Your task:\n"
    "1. Extract the chronological sequence of events from the source material.\n"
    "2. Output a valid Mermaid.js flowchart using `graph TD` syntax.\n"
    "3. After the Mermaid block, provide a brief plain-English summary of the timeline.\n\n"
    "STRICT Mermaid formatting rules: graph TD, camelCase node IDs, short labels (8-10 words), "
    "dates at the start, edge labels for detail, decision nodes in curly brackets, "
    "subgraphs to group phases, all labels double quoted, no `end` as node ID, 5-12 nodes total."
)
```

SYNTHESIS_SYSTEM

```
SYNTHESIS_SYSTEM = (
    "You are an expert Australian Property Law tutor providing a final answer "
    "to a student's question.\n\n"
    "GROUNDING RULES:\n"
    "- FACTS must come from the sources: all case names, dates, statutory "
    "references, legal tests, and holdings you mention MUST appear in the "
    "provided source material or upstream IRAC/chronology analysis.\n"
    "- EXPLANATIONS are encouraged.\n"
    "- Use inline citations: (Source: Readings Week 3) or (Source: Lecture 2 Slide 5).\n"
    "- If sources are insufficient, say so.\n"
    "- If CONVERSATION HISTORY is provided, continue the same study session.\n\n"
    "CRITICAL, MERMAID DIAGRAM PRESERVATION RULE:\n"
    "If the DERIVED ANALYSIS section contains a ```mermaid code block, you MUST "
    "copy that exact ```mermaid ... ``` block verbatim into your final answer."
)
```

VERIFICATION_REWRITE_SYSTEM

```

VERIFICATION_REWRITE_SYSTEM = (
    "You are a careful Property Law editor. You will be given:\n"
    "1. The student's question.\n"
    "2. The retrieved source material.\n"
    "3. A draft answer.\n"
    "4. A list of CASE CITATIONS in the draft that DO NOT appear in the sources.\n\n"
    "Your task: rewrite the draft answer so that EACH unsupported citation is "
    "either (a) removed entirely, or (b) hedged with a phrase that signals it is "
    "not in the provided materials. Preserve every supported claim, every Markdown "
    "heading, and any ```mermaid code block VERBATIM. Do not introduce new "
    "citations. Do not mention this rewrite process to the student. Return ONLY "
    "the rewritten answer."
)

```

MEGA_PROMPT_SYSTEM, the ablation

```

MEGA_PROMPT_SYSTEM = (
    "You are an expert Australian Property Law tutor. Produce a response with "
    "the sections below.\n\n"
    "### SECTION 1, IRAC ANALYSIS\n"
    "Provide a full IRAC analysis with Issue, Rule, Application, Conclusion. "
    "Start with one-sentence ratio prefixed 'Ratio Decidendi:\n\n"
    "### SECTION 2, CHRONOLOGICAL FLOWCHART (conditional)\n"
    "If the query asks for a timeline, output a valid Mermaid.js graph TD diagram.\n\n"
    "### SECTION 3, PLAIN-ENGLISH SUMMARY\n"
    "Concise plain-English explanation with inline citations.\n\n"
    "GROUNDING RULES (apply to all sections): all case names, dates, statutory "
    "references, legal tests, and holdings MUST appear in the provided source material."
)

```

The conditional in section 2 is exactly what fails. The model often produces a Mermaid block on factual queries because the prompt structure encourages all three sections to fire. This is the over triggering that the report calls out.

JUDGE_GROUNDEDNESS_SYSTEM

```

JUDGE_GROUNDEDNESS_SYSTEM = (
    "You are an impartial evaluator assessing the groundedness of an AI answer.\n\n"
    "Score the answer on a scale of 1 to 5:\n"
    " 5 = Fully grounded: every factual claim supported by sources.\n"
)

```

```
" 4 = Mostly grounded: core claims supported, minor unsupported details.\n"
" 3 = Partially grounded: some claims supported, some not.\n"
" 2 = Mostly ungrounded.\n"
" 1 = Ungrounded.\n\n"
"Note: paraphrasing and plain-English explanations are ALLOWED. Only penalise "
"for invented factual claims (case names, dates, statutes, holdings).\n\n"
"Respond with ONLY a JSON object: {\"score\": , \"reasoning\": \"\"}"
)
```

JUDGE_CRITIQUE_SYSTEM

```
JUDGE_CRITIQUE_SYSTEM = (
    "You are a senior evaluator reviewing a junior judge's assessment.\n\n"
    "Either AGREE with the score if the reasoning is sound, or OVERRIDE with a "
    "corrected score if the junior judge was too lenient or too harsh.\n\n"
    "Paraphrasing and plain-English explanations are ALLOWED. Only penalise for "
    "invented factual claims.\n\n"
    "Respond with ONLY a JSON object: {\"score\": , \"reasoning\": \"<>\", "
    "\"agreed_with_draft\": }"
)
```

SLIDE_DESCRIPTION_PROMPT, used at index time by the VLM

```
SLIDE_DESCRIPTION_PROMPT = (
    "You are a legal education assistant. Describe this property law lecture "
    "slide in detail. Include all visible text verbatim, any diagrams or "
    "flowcharts, legal principles, case names, statutory references, and "
    "definitions. Preserve the structure (headings, bullet points) as closely "
    "as possible."
)
```

Every Helper, Every Default

This section enumerates every private function, every fallback path, and every default value in the codebase. If an interviewer points at a helper and asks why, the answer is here.

Indexer helpers in `src/indexing/build_index.py`

Function	What it does and why
<code>_week_sort_key</code>	Pulls the integer out of a folder name like <code>week_3</code> using a regex. Sorts <code>week_2</code> before <code>week_10</code> .
<code>_sanitise_id</code>	Replaces every character that is not alphanumeric, underscore, or hyphen with an underscore. ChromaDB has rules about ID characters.
<code>_classify_pdf</code>	Looks at the lowercased filename. "reading" in the name produces reading. "tutorial" in the name produces tutorial. Anything else is supplementary.
<code>extract_text_from_pdf</code>	Opens the PDF with PyMuPDF, joins every page text with double newlines, closes the document.
<code>describe_slide</code>	Reads the image bytes, builds a Gemini Part.from_bytes with the right MIME type, calls generate_content with the slide description prompt.
<code>_ingest_pdfs</code>	Iterates every PDF in a directory, chunks the text, builds deterministic IDs and metadata dicts, calls collection.add_texts.
<code>_ingest_slides</code>	Iterates every JPEG or PNG, calls the VLM, skips empty descriptions, indexes one slide as one chunk with image_path set.
<code>MIME_TYPES</code>	Tiny dict mapping .jpg and .jpeg to image/jpeg and .png to image/png.

Embedding helpers in `src/embeddings.py`

Function	What it does
<code>__init__</code>	Reads ZEMBED_API_KEY from env if not passed. Raises a clear

Function	What it does
	error if missing. Sets up the Authorization header once.
<code>_embed</code>	Posts JSON to the ZeroEntropy embed endpoint with a 120 second timeout. Treats any non OK response as a runtime error.
<code>embed_documents</code>	LangChain hook used during indexing. Calls <code>_embed</code> with <code>input_type="document"</code> . Short circuits on empty input.
<code>embed_query</code>	LangChain hook used during retrieval. Calls <code>_embed</code> with <code>input_type="query"</code> and unwraps the single result.

Retrieval helpers in **src/agent/tools.py**

Function	What it does
<code>_get_vectorstore</code>	Lazy singleton. Builds a Chroma persistent client and wraps it with the LangChain Chroma store on first call.
<code>_build_filter</code>	Returns one of three shapes. None when there are no constraints. A single condition dict when there is one. An <code>\$and</code> wrapper when both week and type apply.
<code>_search</code>	Dispatches between MMR and similarity. Defaults to <code>RETRIEVAL_STRATEGY</code> from config but accepts an override for ablation experiments.
<code>_raw_to_retrieved</code>	Translates LangChain Document objects to the codebase's <code>RetrievedDocument</code> shape.
<code>_maybe_rerank</code>	If <code>use_reranker</code> is True, imports <code>get_reranker</code> , calls <code>rerank(query, docs, top_k=k)</code> , returns the truncated list. Otherwise truncates the input to <code>top_k</code> directly. Wrapped in try except so a missing reranker degrades to MMR order.
<code>retrieve_texts</code>	Fixes the doc type filter to <code>["reading", "tutorial",</code>

Function	What it does
	"supplementary"]. Picks <code>fetch_k = RERANKER_FETCH_K_TEXT (16)</code> when reranking, else <code>k</code> . Calls <code>_search</code> then <code>_maybe_rerank</code> .
<code>retrieve_slides</code>	Fixes the doc type filter to <code>["lecture_slide"]</code> . Picks <code>fetch_k = RERANKER_FETCH_K_SLIDES (8)</code> when reranking, else <code>k</code> .
<code>retrieve_all</code>	Calls both functions and returns a tuple. The two halves stay separate in the agent state.

Reranker helpers in **src/agent/reranker.py**

Function	What it does
<code>get_reranker</code>	Module level lazy singleton. First call constructs the <code>CrossEncoderReranker</code> wrapper. Every subsequent call returns the cached instance.
<code>CrossEncoderReranker.__init__</code>	Stores the model name, sets <code>_model</code> to <code>None</code> . The actual SDK import and model load is deferred.
<code>_ensure_loaded</code>	On first call, imports <code>CrossEncoder</code> from <code>sentence_transformers</code> and constructs the model. Raises a clear <code>ImportError</code> if the package is missing.
<code>rerank</code>	Builds (query, content) string pairs, calls <code>predict</code> , sorts descending by score, truncates to <code>top_k</code> . Wrapped in <code>try except</code> so any failure returns the original order.

Verification helpers in **src/agent/verification.py**

Function	What it does
<code>_CASE_NAME_PART</code>	Regex fragment that matches a single case name token plus up to four extra capitalised tokens, with stop words

Function	What it does
	like "of", "the", "and" allowed between them.
<code>_CASE_PATTERN</code>	Compiled regex that matches a citation in X v Y or X vs Y shape, tolerating italics, underscores, periods after the separator.
<code>_LEFT_TRIM_PREFIXES</code>	Set of sentence initial words ("In", "The", "A", etc) that should be stripped from the left party name to avoid false captures.
<code>_trim_leading_stopwords</code>	Drops leading stopwords from a captured left side so "In Buckinghamshire" becomes "Buckinghamshire".
<code>extract_case_citations</code>	Returns a unique, order preserving list of normalised X v Y strings extracted from the input text.
<code>_normalise</code>	Lowercase, collapse whitespace, strip italic markers. Used on both source text and candidate citation before substring matching.
<code>case_appears_in_sources</code>	Returns True if either the full normalised X v Y string is present, or both party tokens occur individually.
<code>find_unsupported_cases</code>	Combines extract and check. Returns the citations from the answer that do not appear in the sources.

Node helpers in **src/agent/nodes.py**

Function	What it does
<code>_append_trace</code>	Returns a new list with the current node name appended. Crucially does not mutate the original list.
<code>_format_context</code>	Builds the PRIMARY EVIDENCE block shown to the ratio extractor, chronology node, synthesis, and verification rewrite. Two sections, one per modality, each chunk

Function	What it does
	numbered and labelled with its source, week, and type.
Router parse fallback	If json.loads fails on the router output, the intent defaults to summary. Erring toward giving more, not less.
Router intent validation	After parsing, the intent is checked against {"ratio", "chronology", "summary", "general"}. Anything else is coerced to summary.
Week filter precedence	existing_week if existing_week else parsed_week. Caller wins.
Ratio sentence extraction	Scans output lines for both words "ratio" and "decidendi". Falls back to first non empty line.
Chronology Mermaid extraction	Regex captures the first ```mermaid ... ``` block. Empty string if missing.
Chronology summary extraction	Regex captures everything after Timeline Summary: . Case insensitive.
_collect_sources_text	Verification helper. Concatenates the content and source name of every retrieved text and slide chunk into one big string for substring matching.

Chat memory helpers in **src/agent/chat_memory.py**

Function	What it does
	Drops messages with bad roles or empty content.
prepare_chat_history_for_run	Truncates any single message over 3500 chars. Keeps the most recent 24 messages.
format_transcript_for_llm	Renders the kept history as numbered Student or Tutor lines joined with double newlines.

Function	What it does
build_retrieval_query	If history is empty, returns the current query. Otherwise walks history backwards looking for the last assistant turn, takes a 1200 char excerpt, and prepends it inside a labelled wrapper.
get_chat_history	Safe accessor. Returns an empty list if chat_history is missing or not a list.

LLM dispatcher helpers in **src/llm.py**

Function or default	What it does
_get_gemini, _get_openai, _get_anthropic	Each one is a lazy singleton that builds the SDK client on first call and caches it.
_call_gemini	Wraps the system instruction in a GenerateContentConfig. Reads usage_metadata for token tracking.
_call_openai	Builds a list of input messages with the system instruction first. Optionally attaches a reasoning effort hint. Catches exceptions whose text contains "reasoning" and retries without the hint.
_call_anthropic	Hard codes max_tokens=4096. System prompt sits in the top level system field.
_REASONING_EFFORT dict	Maps gpt-5.3 family to medium effort. Newer GPT-5.4 family is mapped to None so no hint is sent.
TokenUsage.record	Appends a per call dict and increments running totals.
reset_token_usage	Clears the global tracker. Called between every eval case.

Eval helpers in `src/eval/run_evals.py`

Function	What it does
<code>_eval_case_last_turn</code>	Returns the final user turn for a case. Used as the baseline input and the relevancy judge question.
<code>_judge_question_for_case</code>	For multi turn cases, builds a wrapper that says "score the final answer in light of the whole thread" and lists prior turns.
<code>_cross_modal_signals</code>	For cross modal cases, records the count of text and slide chunks retrieved and a flag for whether both modalities fired.
<code>_run_agent_turns_for_eval</code>	For multi turn cases, runs the agent once per turn, accumulating chat history between turns. Accepts <code>use_reranker</code> for the no reranker ablation.
<code>_run_ablation_turns_for_eval</code>	Same as above but for the mega prompt ablation.
<code>_judge_call</code>	Centralised judge invocation. Always temperature 0. Strips optional code fences and parses JSON.
<code>_ranking_metrics_from_binary_verdicts</code>	Computes MRR, Hit at K, NDCG at K from a list of booleans in production order.
<code>judge_context_precision_with_pool</code>	Optional deeper retrieval. Fetches a larger pool, judges every chunk once, computes recall vs pool.
<code>_extract_mermaid_from_text</code>	Same regex as the chronology node. Used to find a Mermaid block in any string.
<code>_mermaid_for_structural_metric</code>	Prefers the dedicated chronology output. If

Function	What it does
	absent, falls back to whatever Mermaid block sits inside the final answer.
<code>_irac_for_structural_metric</code>	Same logic for IRAC.
<code>check_mermaid_validity</code>	Five binary checks. Score is the fraction passed.
<code>check_irac_compliance</code>	Four binary checks for Issue, Rule, Application, Conclusion. Returns {"applicable": False, "score": None} for chronology and general intents.
<code>_run_node</code>	Tiny timing wrapper. Returns the node's result plus elapsed seconds rounded to two decimals.
<code>run_agent_with_metrics</code>	The eval version of <code>run_query</code> . Calls each node directly so per node latencies can be measured. Branches based on intent. Always runs the verification node when <code>USE_VERIFICATION</code> is on. Accepts <code>use_reranker</code> override for the no reranker ablation.
<code>run_baseline</code>	One <code>llm_call</code> with <code>BASELINE_MODEL</code> . No retrieval. No chat history.
<code>run_ablation_mega_prompt</code>	Pre seeds <code>intent="summary"</code> , runs retrieval, then runs <code>mega_prompt_node</code> . Skips the LangGraph compile entirely.
<code>_VerificationCounters</code>	Module scoped live counter. Records cases seen, fired count, rewrites applied, and unsupported total. Logs an INFO line after

Function	What it does
	every case.
<code>_compute_summary</code>	Aggregates the per case results into <code>eval_summary.json</code> . Computes averages, captures per query lists, builds the by family block, includes the four configurations.
<code>_summary_by_query_family</code>	Groups results by family. Adds a fraction with both modalities for cross modal.
<code>_generate_failure_analysis</code>	Writes the Markdown failure analysis. Includes the reranker contribution table comparing the agent and the no reranker ablation per query.
<code>_generate_plots</code>	Plot generator. Includes the four way groundedness comparison and the precision with vs without reranker chart.

Streamlit helpers in **app.py**

Function or block	What it does
<code>render_message</code>	Splits content on the Mermaid fence. Plain segments to <code>st.markdown</code> . Mermaid segments to <code>st_mermaid</code> .
Mermaid append guard	If <code>mermaid_diagram</code> is set in state but the final answer does not contain a Mermaid fence, the UI appends one before storing the message.
Trace banner	Three Streamlit metric widgets show intent, latency, and the node trace as an arrow joined string.
Sources expander	Lists every retrieved text and slide with its week, type, source

Function or block What it does

filename, and content length.

Decisions and Trade Offs

Every non obvious choice in the codebase is here with the reasoning. Each one is phrased as a question an interviewer might actually ask.

Why a multi node graph instead of a single careful prompt

The hypothesis itself is the answer. Cognitive separation against attention dilution. The mega prompt ablation tests this directly with the same retrieval bundle. The agent gained 0.09 groundedness points and 9 percentage points of Mermaid validity over the ablation. The only difference between them is the LangGraph state machine, so those gains attach to node specialisation, not to better evidence or stronger models.

Why MMR plus a cross encoder instead of just MMR or just BM25

Legal text contains many near duplicate paragraphs across readings, slides, and tutorials. Plain similarity returns ten variations of the same passage. MMR fetches a diverse candidate pool. The cross encoder then re scores that pool against the actual question, which lifts the most relevant chunks above the diversity driven candidates. Using both gives you a diverse pool first and pure relevance ranking inside it second. Hybrid BM25 plus dense remains the next future work item to lift exact statute and case name surface forms.

Why a verification node when synthesis is already grounded

Synthesis is told strictly to ground every claim. Models drift anyway. The verification node is a cheap regex sweep over the final answer that catches case citations missing from the retrieved chunks. Across 22 cases it fired 22 times and intercepted zero unsupported citations, which means the synthesis prompt is doing its job today, but the safety net stays in place for the day a model regression sneaks an out of context case into a final answer.

Why 1500 character chunks with 300 character overlap

Legal text is dense. A single ratio holding can span half a page. 1500 characters preserves the typical paragraph plus a sentence of context on either side. The 300 character overlap protects against splits that fall in the middle of a key clause.

Why over fetch 16 text and 8 slide candidates instead of 8 and 4 directly

The cross encoder needs a candidate pool to rerank. Without over fetch the reranker has nothing to choose from. The over fetch creates a diverse pool from MMR, then the cross encoder re scores and truncates to the production sized 8 plus 4. This is what lifts context precision from 0.59 to 0.66 in the eval.

Why one Chroma collection instead of one per modality

Two reasons. First, retrieval has to sometimes return both modalities for the same query, which is simpler with metadata filters than with two collections and a merge step. Second, the multimodal eval treats slides and texts on the same footing, and using one collection means the same MMR pass and the same embedding model apply to both.

Why describe slides with a VLM at index time instead of querying images at runtime

Indexing once is cheap relative to querying every time. The VLM produces a verbose description that captures every visible piece of text, every diagram, and every case name verbatim. Cross modal retrieval hit both_modalities=True on every cross modal case, which validates this design.

Why summary fires both reasoning nodes

A query like "explain X and give me the sequence of events" needs both an IRAC and a Mermaid diagram. Routing to chronology alone produces a bare diagram with no analysis. Routing to ratio alone produces analysis with no timeline. Summary is the path that fires both.

Why per node model assignments

Each node has a different cognitive complexity. Routing is classification and tolerates a cheap model. Chronology is structurally constrained and tolerates a small model. Ratio extraction needs structured legal reasoning and uses a mid tier model. Synthesis needs the strongest grounded reasoning. Per node assignments mean cheap models do cheap work.

Why three providers

To break same model self evaluation bias. The agent uses Gemini in some nodes and OpenAI in others. The judge uses Gemini for the draft and OpenAI for the critique.

Why a two stage groundedness judge

A single LLM judge inherits all the blind spots of its provider. Two stages with two providers act as a check and balance. The eval JSON keeps both scores so the disagreement rate can be inspected. The five case human spot check shows zero disagreement on this dataset, which is a sanity check on the judge framework.

Why intent conditional IRAC scoring

Without it, the agent routed correctly to chronology on a "show me a flowchart" query and was penalised for not producing an IRAC, even though IRAC was never asked for. The conditional helper returns "not applicable" for chronology and general intents, the summary excludes those scores from the average, and the report quotes the applicable case count alongside the average for transparency.

Why total equals false on AgentState

It lets every key be optional at graph entry. A new state can be passed in with just a query. Each node only writes the keys it produces.

Why a packed retrieval query for follow ups

Embedding the entire prior tutor reply washes out the new student question. A 1200 character excerpt of the prior reply gives the embedding enough topic anchor without dominating the vector. After the embedding goes through MMR the cross encoder re scores against the packed query, which keeps the final pool tight on the actual follow up topic.

Why the synthesis prompt mandates a verbatim Mermaid block

Because the model would otherwise paraphrase the diagram into bullet points, and the Streamlit UI cannot render bullet points as a diagram. Mermaid only renders if the exact fenced block reaches the rendered message. Three guards: the system prompt, the user prompt, and the frontend append guardrail.

Why default to summary on router parse failure

Summary fires both reasoning nodes. If we default to general the user gets the simplest possible answer. If we default to summary the user gets the most complete answer.

Why temperature 0.2 for nodes and temperature 0 for judges

Reasoning needs slight diversity for prose flow. Judges should be reproducible across runs.

Why per node latency tracking

Because the design hypothesis claims a cost. The 0.09 groundedness gain costs the user 14 extra seconds versus the mega prompt. Per node latencies make that cost concrete. Chronology at 8.6s and ratio extractor at 6.6s dominate.

Why recall vs pool is a proxy and not real recall

Real recall needs a fully labelled relevant set across the whole corpus. The eval has no human qrels. Recall vs pool uses the relevant chunks anywhere in the enlarged retrieval pool as a stand in.

Why the no reranker configuration is a separate full run instead of a metadata flag

Because the eval needs to score the same metrics on the same case with the cross encoder disabled. Re running the agent with `use_reranker=False` via the state override produces a clean fourth column in the results that isolates the reranker's contribution.

Test Suite

Tests live in tests/ and are organised by phase. Two categories. Unit tests have no marker and run without API keys. Integration tests carry `@pytest.mark.integration` and slow ones additionally carry `@pytest.mark.slow`. Both markers are declared in `pyproject.toml`.

File	Phase	What it covers
test_phase1_indexing.py	Indexing	Config sanity. PDF text extraction. ZeroEntropy embedding integration. ChromaDB collection integrity and metadata fields.
test_phase2_retrieval.py	Retrieval and state	State TypedDict accepts every key. Filter builder produces the right shapes. Real retrieval pulls back text and slides, week filter holds. Reranker can be disabled via <code>use_reranker=False</code> .
test_phase3_nodes.py	Nodes	Trace appending without mutation. Context formatting. Live router classification. Live ratio extractor produces IRAC. Live chronology produces Mermaid. Verification node fact checks citations.
test_phase4_graph.py	Graph	Graph compiles. All six nodes present when verification is enabled. Cached singleton stays the same instance. Conditional path function returns the right next node. End to end ratio, chronology, summary, general paths produce the expected node trace including verification.
test_phase5_eval.py	Evaluation	Ranking metric edge cases. Eval cases cover all four families. Compute summary handles

File	Phase	What it covers
		four configurations. Plot generation produces every expected PNG including the four way comparison.
test_chat_memory.py	Memory helpers	Sanitisation drops invalid roles. Transcript formatting is numbered. Retrieval query stays standalone with no history. Retrieval query packs an excerpt on follow ups.
test_verification.py	Citation extraction	The case regex tolerates italics. Trim leading stopwords removes "In", "The", and similar. Two token fallback catches references where the source quotes the parties without the v separator.

Running them

```
pytest tests/ -v
pytest tests/ -v -m "not integration" # quick suite, no API keys
```

File by File Reference

Quick map from the question "where does X live" to the file you should open.

File	Purpose
app.py	Streamlit chat UI. Sidebar, message history, render helper, trace banner, source expanders.
src/config.py	All tunable knobs. Paths, embedding model, per node model assignments, chunk sizes, MMR settings, reranker fetch sizes, USE_RERANKER and USE_VERIFICATION flags, chat memory

File	Purpose
	caps, eval pool sizes.
src/llm.py	Multi provider dispatcher. llm_call, lazy clients, token tracking, per provider quirks.
src/embeddings.py	ZeroEntropy zembed-1 wrapper that follows the LangChain Embeddings contract.
src/indexing/ build_index.py	Offline indexer. Walks data/, extracts PDF text, describes slides via VLM, writes Chroma.
src/agent/state.py	Typed dicts for state, retrieved documents, and chat messages. Includes use_reranker override field and verification_report output field.
src/agent/ chat_memory.py	Sanitise history, render transcripts, build packed retrieval query, safe history reads.
src/agent/tools.py	Vector store singleton, filter builder, MMR vs similarity dispatch, retrieve_texts, retrieve_slides, retrieve_all, the _maybe_rerank helper that calls the cross encoder.
src/agent/ reranker.py	Cross encoder reranker. Module level lazy singleton plus the CrossEncoderReranker wrapper.
src/agent/ verification.py	Citation regex, normalisation, source containment check, find_unsupported_cases.
src/agent/nodes.py	The six cognitive nodes including verification. Plus the mega prompt ablation node.
src/agent/graph.py	Graph builder, conditional edge functions, cached compile, run_query. Wires verification after synthesis when USE_VERIFICATION is on.
src/eval/ run_evals.py	Eval cases (22 cases across 4 families), four configurations, judges, ranking metrics, structural checks (intent conditional

File	Purpose
	IRAC), summary builder, plot generator, failure analysis report, verification counter.
tests/	Phase based tests. Unit and integration markers.
docs/ architecture.md	Architecture overview written in Mermaid and tables.
requirements.txt	Pinned libraries including sentence-transformers.
pyproject.toml	Pytest configuration. Markers and warning filters.
chroma_db/	Persistent local vector store. Created by the indexer.
eval_results/	Output of the eval suite. JSON, Markdown failure analysis, human spot check log, PNG plots.

Configuration cheat sheet

Every value in src/config.py

```
DATA_DIR = Path("data")
CHROMA_DIR = Path("chroma_db")

EMBEDDING_MODEL = "zembed-1"
EMBEDDING_DIMENSIONS = 2560

VLM_MODEL = "gemini-2.5-pro"

ROUTER_MODEL = "gemini-3.1-flash-lite-preview"
RATIO_EXTRACTOR_MODEL = "gpt-5.3-chat-latest"
CHRONOLOGY_MODEL = "gemini-3-flash-preview"
SYNTHESIS_MODEL = "gpt-5.4-mini"

BASELINE_MODEL = "gpt-5.4-mini"
ABLATION_MODEL = "gpt-5.4-mini"

JUDGE_DRAFT_MODEL = "gemini-3-flash-preview"
```

```
JUDGE_CRITIQUE_MODEL = "gpt-5.4"

CHROMA_COLLECTION = "property_law_kb"

CHUNK_SIZE = 1500
CHUNK_OVERLAP = 300

RETRIEVAL_STRATEGY = "mmr"
MMR_LAMBDA = 0.5
MMR_FETCH_K = 20

# Cross encoder reranker (post MMR)
USE_RERANKER = True
RERANKER_MODEL = "cross-encoder/ms-marco-MiniLM-L-6-v2"
RERANKER_FETCH_K_TEXT = 16    # MMR returns 16 text chunks, rerank to 8
RERANKER_FETCH_K_SLIDES = 8   # MMR returns 8 slide chunks, rerank to 4

# Verification node (fact check cited cases against retrieved sources)
USE_VERIFICATION = True

CHAT_HISTORY_MAX_MESSAGES = 24
CHAT_HISTORY_MAX_CHARS_PER_MESSAGE = 3500
CHAT_HISTORY_MAX_ASSISTANT_TAIL_CHARS = 1200

EVAL_RETRIEVAL_POOL_K_TEXT = 20
EVAL_RETRIEVAL_POOL_K_SLIDES = 10
```

Interview Cheat Sheet

Crisp question and answer pairs. Each one is the shortest defensible answer in your own voice.

Q1. WHAT IS THE PROJECT

A multimodal LangGraph agent that helps a law student study Australian property law. Six cognitive nodes including a verification stage. Dual index over PDFs and VLM described slides. Layered retrieval with MMR plus cross encoder. Four configurations evaluated head to head.

Q2. WHAT IS THE HYPOTHESIS

Cognitive separation versus attention dilution. A multi node LangGraph state machine that separates ratio decidendi extraction from chronological fact extraction will produce more accurate causal analyses than a single consolidated mega prompt baseline that receives identical retrieved context.

Q3. WHAT IS THE RESULT

Full agent at 4.86 groundedness, mega prompt at 4.77, no reranker variant at 4.86, baseline at 3.41, on the same 22 cases. The 0.09 groundedness gain over the mega prompt is attributable to cognitive separation. The reranker lifts context precision from 0.59 to 0.66.

Q4. WALK ME THROUGH ONE QUERY END TO END

User message hits Streamlit. `run_query` builds a state. Router classifies intent. Retrieval over fetches via MMR and the cross encoder reranks. The graph forks based on intent. Reasoning nodes write IRAC or Mermaid. Synthesis grounds everything in primary evidence. Verification fact checks every case citation. The UI renders inline.

Q5. WHY LANGGRAPH AND NOT LANGCHAIN

Because the design needs conditional branching after retrieval. Standard chains are linear. A state machine with conditional edges expresses the four routes naturally and lets per node outputs be inspected and ablated independently.

Q6. WHY ZEROENTROPY ZEMBED-1

Asymmetric embeddings tuned for retrieval. Documents and queries are embedded in different modes. 2560 dimensions. Wrapped by hand because LangChain has no built in client.

Q7. WHY MMR WITH LAMBDA 0.5 AND OVER FETCH

MMR balances relevance and diversity. The over fetch creates a 16 text or 8 slide candidate pool that the cross encoder re scores. Diversity comes first via MMR, then pure relevance ranking via the cross encoder.

Q8. WHY A CROSS ENCODER RERANKER ON TOP OF MMR

A bi encoder embeds query and document independently. A cross encoder takes both as a single input and produces a relevance score directly. More accurate but only feasible on a small candidate pool. The hybrid pattern combines bi encoder recall (MMR) with cross encoder precision (rerank).

Q9. HOW DO SLIDES BECOME RETRIEVABLE BY TEXT

Gemini 2.5 Pro reads each slide image at index time and produces a verbose text description. The description is embedded with the same model as the readings. A text query naturally pulls back relevant slides.

Q10. WHAT IS THE FOUR INTENT CLASSIFICATION

Ratio for binding rule and IRAC. Chronology for diagrams alone. Summary when the user wants both an explanation and a timeline. General for greetings or off topic.

Q11. WHY DOES SUMMARY FIRE BOTH REASONING NODES

Because a "explain X and give me the sequence of events" query needs both an IRAC and a Mermaid diagram. The decision rule in the router pushes any combined query to summary.

Q12. WHAT DOES THE VERIFICATION NODE ACTUALLY DO

Extracts every plausible X v Y case citation from the final answer using a permissive regex. Checks each one for full string or two token presence in the concatenated retrieved chunk text. If anything is unsupported, asks the synthesis model to rewrite the answer with that claim removed or hedged. Across 22 eval cases it fired 22 times and intercepted zero unsupported citations.

Q13. WHY IS VERIFICATION SO FAST AT 0 SECONDS

Because the regex check is sub millisecond and the rewrite branch did not trigger across the suite. The expensive call to the synthesis model only happens when a citation needs editing. Otherwise the node is essentially free.

Q14. HOW IS THE MERMAID DIAGRAM PROTECTED FROM BEING LOST

Three guards. The synthesis system prompt mandates a verbatim copy. The user prompt repeats the rule. The Streamlit frontend appends the Mermaid block to the stored message if synthesis omits it. The verification rewrite system prompt also preserves Mermaid blocks verbatim.

Q15. WHAT DOES THE MULTI PROVIDER DISPATCHER DO

Routes llm_call by model name prefix. gemini- goes to Google. gpt- goes to OpenAI Responses API. claude- goes to Anthropic. Lazy clients. Token tracking.

Q16. HOW DOES CONVERSATIONAL FOLLOW UP WORK

Streamlit holds messages in st.session_state. Each new turn passes prior messages as chat_history. The router uses the transcript to resolve "that case". Retrieval prepends a 1200 char excerpt of the prior tutor reply to the embedding query. The reranker then re scores against the packed query.

Q17. WHAT IS THE MEGA PROMPT ABLATION

Same retrieval bundle as the agent. One LLM call replaces every reasoning node and synthesis. Produces all three sections in one pass. Used to measure whether multi node decomposition adds anything beyond a well prompted single call.

Q18. WHY DOES THE MEGA PROMPT SCORE 100% IRAC COMPLIANCE OVER ALL 22 CASES BUT THE AGENT ONLY OVER 19

Because the agent's IRAC scoring is intent conditional. Chronology and general queries are scored not applicable instead of zero. The mega prompt produces all three sections unconditionally, including for queries that should not have IRAC, which is over triggering.

Q19. WHAT IS THE NO RERANKER ABLATION

The full agent run again with use_reranker=False in the agent state. MMR returns the production sized 8 text and 4 slide chunks directly. Same graph, same verification, no cross encoder. Isolates the reranker's contribution.

Q20. WHAT IS THE TWO STAGE GROUNDEDNESS JUDGE

A draft judge from Gemini scores 1 to 5 with reasoning. A critique judge from OpenAI reviews the draft. The critique score is final. Cross provider design reduces same model self evaluation bias.

Q21. HOW DO YOU KNOW THE LLM JUDGE IS RELIABLE

The eval harness writes a five case human spot check log. A human fills in their own scores by hand. Across the five sampled cases the human and LLM judge agreed exactly on every case for both groundedness and relevancy. Five of five exact agreement, MAE 0.0.

Q22. WHAT METRICS DOES THE EVAL COLLECT

Groundedness, answer relevancy, context precision @K, MRR, Hit @K, NDCG @K, optional recall vs pool, source diversity, total latency, per node latency, token usage, Mermaid validity, intent conditional IRAC compliance, cross modal modality flags, verification telemetry.

Q23. WHAT IS RECALL VS POOL AND WHY ISN'T IT REAL RECALL

It divides relevant chunks in the production slice by relevant chunks anywhere in an enlarged retrieval pool. Real recall needs human qrels across the whole corpus. The pool is a bounded proxy.

Q24. WHAT FAMILY HAD THE LOWEST CONTEXT PRECISION AND WHY

Factual retrieval at 0.52. Statute and named test queries miss the exact surface form of the target inside dense embedding space. The fix is hybrid retrieval with BM25 plus dense via reciprocal rank fusion.

Q25. WHAT IS THE AGENT'S BIGGEST WEAKNESS

Retrieval imprecision, not reasoning error. Average context precision is 0.66 and 15 of 22 queries are below 0.70. Per node reasoning is strong enough to filter noise, but the cost is wasted context tokens and elevated latency.

Q26. WHAT IS ON THE FUTURE WORK LIST

Hybrid BM25 plus dense retrieval with reciprocal rank fusion. Streaming and smaller local models for the router and chronology to compress latency. Broader verification beyond case citations to statutes and slide identifiers.

Q27. WHAT DOES **NODE_TRACE LET YOU DO**

Verify which path actually ran, debug routing, surface per node latency, and prove in the eval that the right intent took the right route. Verification appears at the end of every trace when enabled.

Q28. WHAT HAPPENS IF THE CROSS ENCODER FAILS

The `_maybe_rerank` helper wraps the call in try except. On any failure it returns the first top_k documents in MMR order. The graph still runs, retrieval still returns useful chunks, the failure mode degrades to "agent without reranker".

Q29. WHAT HAPPENS IF CHROMA IS EMPTY

Retrieval returns empty lists. The synthesis prompt says explicitly that if sources are insufficient the model should reply with a "consult your additional readings" message rather than fabricate.

Q30. WHAT DOES THE VERIFICATION REPORT LOOK LIKE IN STATE

`{"unsupported_claims": [], "rewrites_applied": False}` on a clean answer.

`{"unsupported_claims": ["Smith v Jones"], "rewrites_applied": True}` when the rewrite branch fired. Both fields are recorded in the eval JSON and surface in the live counter log line.

If they ask you to read a specific file

If they say	You open	Key line numbers
"Show me the graph"	src/agent/graph.py	build_graph wires every node, the conditional edges, and the verification edge.
"Show me the router prompt"	src/agent/nodes.py	ROUTER_SYSTEM at the top of the file.
"How is retrieval done"	src/agent/tools.py	retrieve_all, _search, _build_filter, _maybe_rerank.
"How does the reranker work"	src/agent/reranker.py	get_reranker singleton plus CrossEncoderReranker.rerank.
"How does verification work"	src/agent/verification.py plus verification_node in nodes.py	find_unsupported_cases for the regex sweep, verification_node for the rewrite branch.
"How are slides indexed"	src/indexing/ build_index.py	describe_slide and _ingest_slides.
"How does the dispatcher work"	src/llm.py	llm_call dispatches by prefix.
"How is grounding judged"	src/eval/run_evals.py	judge_groundedness, two stage with draft and critique.
"How are the four configs run"	src/eval/run_evals.py	run_evaluation calls each configuration in turn per case.
"Where is chat memory"	src/agent/ chat_memory.py	Sanitise, format, build retrieval query, safe accessor.

Numbers worth memorising

GroundednessAgent 4.86. No reranker 4.86. Mega prompt 4.77. Baseline 3.41.
Mermaid validityAgent 68%. No reranker 68%. Mega prompt 59%. Baseline 0%.
IRAC complianceAgent 1.00 over 19 applicable. No reranker 1.00 over 18. Mega
prompt 1.00 over 22 (over triggers). Baseline 0.10. Context precisionAgent 0.66.
No reranker 0.59. Reranker contribution +0.07. MRR / NDCGAgent 0.97 / 0.94.
No reranker 0.93 / 0.94. LatencyAgent 23.57s. No reranker 22.11s. Mega prompt
9.52s. Baseline 3.72s. Tokens per queryAgent 26,144. No reranker 27,204. Mega
prompt 10,320. Baseline 357. Per node latencyChronology 8.6s. Ratio 6.6s.
Synthesis 4.6s. Retrieval 2.7s. Router 1.1s. Verification 0.0s. Cross modal
coverage100% of cross modal cases hit both modalities (5 of 5). VerificationFired
22 of 22 cases. Zero unsupported citations. Zero rewrites triggered. Human spot
check5 of 5 exact agreement on both groundedness and relevancy. MAE 0.0.
Suite size22 cases. 6 factual, 5 cross modal, 7 analytical, 4 conversational
including a 3 turn case. Chunk size and overlap1500 characters. 300 character
overlap. Retrieval k8 text plus 4 slide chunks per query. MMR fetch_k 20. Lambda
0.5. Reranker over fetch 16 text and 8 slides. Reranker modelcross-encoder/ms-
marco-MiniLM-L-6-v2, around 80 MB, runs locally. EmbeddingZeroEntropy
zembed-1, 2560 dimensions, asymmetric document or query mode.

Glossary

RATIO DECIDENDI

The binding legal principle the court applied to reach its decision. The single most important sentence in any common law judgment.

IRAC

Issue, Rule, Application, Conclusion. The standard shape used to write up a case in Anglo Australian legal education.

ANIMUS POSSIDENDI

Intention to possess. One of the two limbs of adverse possession alongside factual possession.

MMR

Maximal Marginal Relevance. A re ranking method that balances relevance and diversity. ashGPT uses lambda 0.5 over the top 20 candidates with over fetch.

CROSS ENCODER

A model that takes a query and a document together as a single input and produces a relevance score directly. More accurate than bi encoder dot product, only feasible on a small candidate pool.

BI ENCODER

A model that embeds the query and the document independently and compares with a dot product. ZeroEntropy is a bi encoder.

ASYMMETRIC EMBEDDING

A model that embeds queries and documents in different modes so the vector representations are tuned for retrieval rather than for symmetric similarity.

VLM

Vision Language Model. ashGPT uses Gemini 2.5 Pro to read each slide image and produce a text description that gets embedded.

MERMAID

A text based diagramming syntax. ashGPT generates graph TD flowcharts. The Streamlit UI renders them as interactive SVGs.

LLM AS A JUDGE

Using a language model to grade another language model's output. ashGPT uses cross provider judging plus a critique stage to reduce same model bias.

GROUNDEDNESS

How well an answer's factual claims are supported by the source material it was given.

RECALL VS POOL

A bounded proxy for recall. Relevant chunks in the production slice divided by relevant chunks anywhere in an enlarged candidate pool, judged once.

CONDITIONAL EDGE

A LangGraph edge whose target depends on a small Python function reading the current state. ashGPT uses two of these.

TYPEDDICT

Python typing primitive used to describe the shape of a dict. ashGPT uses `total=False` so all keys are optional.

VERIFICATION NODE

Post synthesis fact check. Extracts case citations from the final answer with a regex, checks each one against the retrieved sources, asks the synthesis model to rewrite the answer with unsupported citations removed.

ATTENTION DILUTION

The failure mode the design hypothesis names. A monolithic prompt asked to produce IRAC plus Mermaid plus synthesis silently trades one structural artefact for another inside one decoder pass.

OVER TRIGGERING

The mega prompt's failure mode. A prompt that always emits every section regardless of whether the question asked for it. Inflates compliance metrics with output the user did not request.

COGNITIVE SEPARATION

The design hypothesis's named pattern. Splitting structurally distinct outputs into specialised nodes so each one is generated under its own constraints rather than under one consolidated directive.

This textbook was generated from the codebase as it stands today. To check

anything against the live source, open the matching file under the path shown in

the file by file reference. Every diagram on this page is rendered live by Mermaid.js.

Every code block is real Python from the repository.